

**PES UNIVERSITY, BANGALORE**

Department of Computer Science and Engineering

JACKFRUIT MINI-PROJECT — UE24CS341A Software Engineering

# **Part-1 Submission: SRS, Software Architecture & Design Specification, and Test Plan**

for

**Simple Shell — POSIX-Compliant Command Line Interpreter in C**

| Item         | Details                                                                                                                             |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Project      | Simple Shell — POSIX-Compliant Command Line Interpreter in C                                                                        |
| Project ID   | SH-2026-10                                                                                                                          |
| Group        | Team 10 (G-10)                                                                                                                      |
| Team Members | Anshul Poovaiah K – PES1UG24CS071 / Archana Shivakumar – PES1UG24CS078 / Ashlesh R V – PES1UG24CS088 / Atishay Jain – PES1UG24CS097 |
| Course       | UE24CS341A – Software Engineering, V Semester                                                                                       |
| Methodology  | Agile (Scrum, 6 sprints)                                                                                                            |
| Version      | Combined submission                                                                                                                 |
| Date         | 3 October 2026                                                                                                                      |

## **Contents of this submission**

| # | Document                                     | Version | Covers rubric item                                                                                                          |
|---|----------------------------------------------|---------|-----------------------------------------------------------------------------------------------------------------------------|
| 1 | Software Requirements Specification          | 1.1     | SRS: descriptive sections, FRs/NFRs, UML use case diagram, security objectives and requirements                             |
| 2 | Software Architecture & Design Specification | 1.0     | Component diagram, architecture pattern, traceability, security architecture, sequence diagrams, API design, error handling |
| 3 | Test Plan (with test cases)                  | 1.0     | IEEE 829 sections, Section 5.1 security validation, 36 test cases, traceability to SRS                                      |

**PES UNIVERSITY, BANGALORE**

Department of Computer Science and Engineering

JACKFRUIT MINI-PROJECT — UE24CS341A Software Engineering

# Software Requirements Specification

for

**Simple Shell — POSIX-Compliant Command Line Interpreter in C**

| Item         | Details                                                                                                                             |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Project      | Simple Shell — POSIX-Compliant Command Line Interpreter in C                                                                        |
| Project ID   | SH-2026-10                                                                                                                          |
| Group        | Team 10 (G-10)                                                                                                                      |
| Team Members | Anshul Poovaiah K – PES1UG24CS071 / Archana Shivakumar – PES1UG24CS078 / Ashlesh R V – PES1UG24CS088 / Atishay Jain – PES1UG24CS097 |
| Course       | UE24CS341A – Software Engineering, V Semester                                                                                       |
| Methodology  | Agile (Scrum, 6 sprints)                                                                                                            |
| Version      | 1.1 (revised for Part-1 submission)                                                                                                 |
| Date         | 3 October 2026                                                                                                                      |

## Revision History

| Name                            | Date        | Reason for Changes                                                                                                                                                                                                                                                                                                                                                                                                                                 | Version |
|---------------------------------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| Archana Shivakumar<br>(Team 10) | 07-Sep-2026 | Initial SRS Phase-1 draft prepared from approved Synopsis                                                                                                                                                                                                                                                                                                                                                                                          | 1.0     |
| Team 10                         | 03-Oct-2026 | Added UML use case diagram and analysis diagrams; added security objectives (SO-1..SO-3); rewrote FRs/NFRs/SECs to be atomic and measurable; added FR-011 (aliases, split from FR-010) and FR-012 (non-interactive mode); added &> and env from the Synopsis scope; renumbered NFRs; added NFR-007, NFR-008, SEC-007; unified job states; completed field layouts and the full Requirement Traceability Matrix; corrected Project ID to SH-2026-10 | 1.1     |

## Table of Contents

1. Introduction
2. Overall Description
3. External Interface Requirements
4. Analysis Models
5. System Features (FR-001 – FR-012)
6. Other Nonfunctional Requirements (Performance, Safety, Security, Quality, Business Rules)
7. Other Requirements

Appendix A: Glossary · Appendix B: Field Layouts · Appendix C: Requirement Traceability Matrix

# 1. Introduction

## 1.1 Purpose

This document specifies the software requirements for Simple Shell, a POSIX-compliant command-line interpreter written in C by Team 10 (Project ID SH-2026-10) for the UE24CS341A Jackfruit Mini-Project. It covers the complete scope of the Simple Shell executable: the interactive REPL, lexical analysis and variable expansion, external command execution, multi-stage pipelines, I/O redirection, job control and signal handling, built-in commands, persistent history, aliases and non-interactive (script) mode. This is Version 1.1 of the SRS; it supersedes Version 1.0.

## 1.2 Intended Audience and Reading Suggestions

- **Team 10 developers** — derive module design and implementation tasks from Section 5 (System Features).
- **Course faculty and evaluators** — verify requirement measurability, traceability and coverage (Sections 5, 6 and Appendix C).
- **Testers** — derive unit, integration and system tests from the numbered FR/NFR/SEC requirements; each requirement names its verification method.
- **Future maintainers** — understand scope and constraints without reading source code first.

Readers interested in what the shell does should read Sections 2 and 5. Readers interested in engineering constraints should read Sections 2.5, 2.6, 6 and 7. Section 4 and Appendix C connect requirements to the companion Software Architecture & Design Specification (SAD) and Test Plan, which are submitted together with this SRS.

## 1.3 Product Scope

Simple Shell is a lightweight, modular command-line interpreter for Linux, built to demonstrate industrial-grade systems-programming practice rather than to replace bash or zsh. It reads input through a Read-Eval-Print Loop, tokenizes and expands it, resolves and executes programs through POSIX system calls, and provides pipelines, redirection, background job control, built-ins, history and aliases. It gives students hands-on experience of process management, IPC and signal handling, and gives Interactive Users and Script Runners (Section 2.3) a small, auditable, memory-safe interpreter. Scripting control structures (if/for/while, functions), a GUI, networking and a line-editing library are out of scope.

## 1.4 References

- PES University, Dept. of CSE — Jackfruit Mini-Project: Software Engineering Project Guidelines, UE24CS341A, 2026.
- Team 10 — Simple Shell (CLI): Deliverable 1 — Synopsis / Project Proposal, v1.0 (approved).
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications (basis of the course SRS template).
- The Open Group Base Specifications Issue 7 (POSIX.1-2008) — fork, execvp, pipe, dup2, sigaction, waitpid, tcsetpgrp.
- GNU Bash Reference Manual — black-box behavioural comparison baseline used in verification.
- Team 10 — Software Architecture & Design Specification v1.0 and Test Plan v1.0 (companion documents).

## 2. Overall Description

### 2.1 Product Perspective

Simple Shell is a new, self-contained product. It runs as an ordinary user-space process directly above the Linux kernel, does not modify the kernel and coexists with the system's login shell. Its only runtime dependencies are the POSIX system-call interface and the standard C library (glibc). Figure 4.1 shows its position between the user/script and the kernel-managed processes and files it orchestrates.

### 2.2 Product Functions

- Display a prompt and read command lines interactively (REPL), or read them from a script without a prompt.
- Tokenize input honouring quotes and escapes; expand \$VAR, \$? and aliases.
- Execute any program found on PATH as a child process and report its exit status.
- Chain commands into pipelines with |.
- Redirect stdin, stdout and stderr to and from files (<, >, >>, 2>, &>).
- Run commands in the background with & and manage them with jobs, fg and bg; forward Ctrl+C / Ctrl+Z to the foreground job.
- Run built-ins (cd, pwd, echo, export, unset, env, exit, jobs, fg, bg, history, alias, unalias) inside the shell process.
- Keep command history across sessions and support user-defined aliases.

### 2.3 User Classes and Characteristics

| User Class                   | Frequency / Privilege                        | Characteristics                                                                                                                              |
|------------------------------|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Interactive Developer / User | Frequent; normal (non-root) user privileges  | Uses the shell interactively; relies on pipelines, redirection, Ctrl+Z job suspension and history. Highest-priority class.                   |
| Script / Automation Runner   | Frequent, non-interactive; normal privileges | Feeds command sequences via stdin or a script file. Needs deterministic exit codes and fast start-up; does not use history or prompts.       |
| Systems Evaluator / Faculty  | Occasional; audit access                     | Exercises boundary conditions (deep pipelines, malformed input, permission-denied files) and reviews Valgrind/ASan/static-analysis evidence. |

### 2.4 Operating Environment

- Ubuntu 22.04 LTS or Debian 12, Linux kernel 5.15 or later.
- C11, GCC 11+ with -std=c11 -Wall -Wextra -Werror -pedantic; GNU Make.
- Any POSIX terminal or pseudo-terminal; no GUI dependency.
- Docker images of Ubuntu 22.04 and Debian 12 for reproducible CI builds, grading and demo.

### 2.5 Design and Implementation Constraints

- **Process:** Agile/Scrum with GitHub (and GitHub Projects or Jira) used for backlog and sprints; every member commits implementation code.
- **Language:** C with POSIX system calls only; no Python/Lua runtimes or third-party shell libraries in the shipped binary.
- **Memory:** all dynamic allocations freed; zero leaks and invalid accesses are acceptance criteria (NFR-002).
- **File descriptors:** every pipe end not needed by a process is closed in that process.

- **Signal handlers:** call only async-signal-safe functions (e.g., write, waitpid); no printf or malloc inside handlers.
- **CI:** GitHub Actions (or Jenkins) runs build, Cppcheck/Clang-Tidy, unit tests, integration tests and Valgrind on every pull request.

## 2.6 Assumptions and Dependencies

- The host provides a POSIX-compliant kernel and glibc toolchain; Simple Shell does not install them.
- Interactive job-control tests are run in a real terminal or pty, because SIGTSTP/terminal control are unavailable when stdin is not a TTY.
- Development-only tools (Unity/Criterion, Valgrind, Cppcheck, Docker, expect) are not linked into the shipped binary.
- External programs launched by the shell (ls, grep, cat, etc.) are installed on PATH; their correctness is not the shell's responsibility.
- If GitHub Actions quotas or board access change mid-project, tracking moves to an equivalent tool; this is a project risk, not a requirements change.

## 3. External Interface Requirements

### 3.1 User Interfaces

- **Prompt format:** `user@hostname:cwd$` (the home directory is shown as `~`), rendered before each interactive input line.
- **Line input:** read with `getline()`; no arrow-key editing or history recall keys (line-editing library is out of scope). History is accessed with the `history` built-in.
- **Error messages:** every shell-generated error starts with `simple_shell:` and is written to `stderr`, never `stdout` (NFR-007).
- **Exit status convention (\$?):** 0 success; 1 general/redirection error; 2 syntax/usage error; 126 found but not executable; 127 command not found; 128+n terminated by signal `n`.

### 3.2 Software Interfaces

| Interface                                        | Direction                                             | Purpose                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Linux kernel (POSIX system calls)                | Bidirectional                                         | <code>fork</code> , <code>execvp</code> , <code>pipe</code> , <code>dup2</code> , <code>waitpid</code> , <code>sigaction</code> , <code>setpgid</code> , <code>tcsetpgrp</code> , <code>kill</code> , <code>open</code> , <code>close</code> , <code>chdir</code> — the shell's whole execution and I/O model. |
| File system: <code>~/simple_shell_history</code> | Read at start-up, written at exit                     | Persistent history; mode 0600 (SEC-006); opened with <code>O_CLOEXEC</code> (SEC-007).                                                                                                                                                                                                                         |
| File system: redirection targets                 | Read / write                                          | Opened with the invoking user's permissions (SEC-003).                                                                                                                                                                                                                                                         |
| Child programs                                   | <code>stdin/stdout/stderr</code> via file descriptors | The shell only wires descriptors before <code>execvp</code> and collects exit status; it never interprets child output.                                                                                                                                                                                        |
| Unit-test framework (Unity/Criterion)            | Build-time only                                       | Links against internal modules for white-box testing; not present in the release binary.                                                                                                                                                                                                                       |

### 3.3 Communications Interfaces

No network communication is required. The only channels are local standard streams and POSIX signals (`SIGINT`, `SIGTSTP`, `SIGCHLD`, `SIGCONT`, `SIGTTOU`) delivered by the kernel.

### 3.4 Hardware Interfaces

Hardware is accessed only indirectly through the kernel's TTY/pty layer. A controlling terminal is needed for `Ctrl+C` (`SIGINT`) and `Ctrl+Z` (`SIGTSTP`); when input is not a TTY, the shell runs in non-interactive mode (FR-012) with job control disabled.

# 4. Analysis Models

## 4.1 System Context Diagram

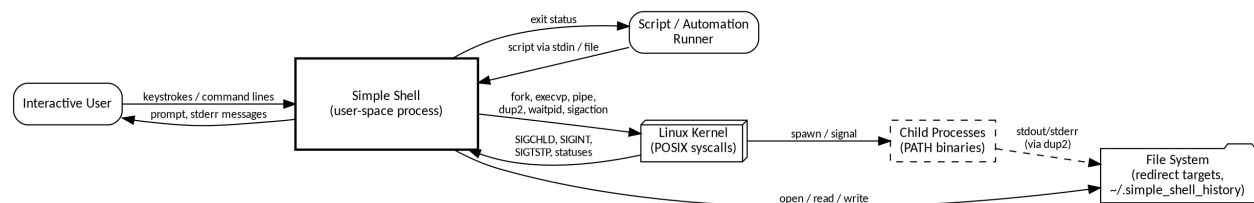


Figure 4.1 – System context of Simple Shell

| Actor                      | Goal / Need                                         | Major Actions                                                                  |
|----------------------------|-----------------------------------------------------|--------------------------------------------------------------------------------|
| Interactive User           | Run utilities, chain tools, manage jobs             | Type commands; pipe and redirect; suspend/resume jobs; use history and aliases |
| Script / Automation Runner | Run command sequences with deterministic exit codes | Pass a script file or stdin; read final exit status                            |
| Linux Kernel (supporting)  | Create and manage processes and descriptors         | fork/exec, pipe buffers, deliver SIGINT/SIGTSTP/SIGCHLD                        |

## 4.2 Use Case Model

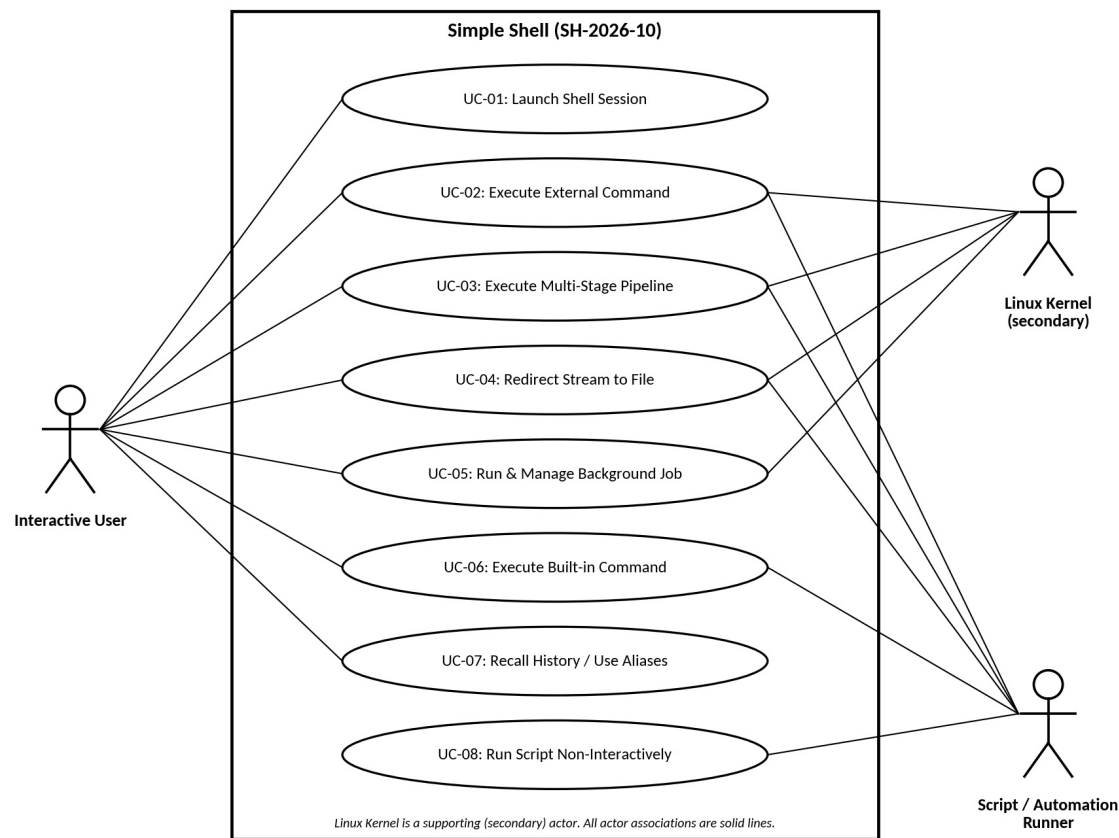


Figure 4.2 – UML use case diagram

| ID    | Use Case                      | Primary Actor        | Precondition                               | Main Success Outcome                             | FRs                    |
|-------|-------------------------------|----------------------|--------------------------------------------|--------------------------------------------------|------------------------|
| UC-01 | Launch Shell Session          | Interactive User     | Binary is executable                       | Prompt shown; history loaded                     | FR-001, FR-010         |
| UC-02 | Execute External Command      | User / Script Runner | Shell running                              | Program run; output shown; \$? set               | FR-002–FR-004          |
| UC-03 | Execute Multi-Stage Pipeline  | User / Script Runner | Shell running; $\geq 2$ commands joined by | Stages connected; status of last stage in \$?    | FR-005                 |
| UC-04 | Redirect Stream to File       | User / Script Runner | Shell running; target path given           | Stream read from / written to file               | FR-006                 |
| UC-05 | Run and Manage Background Job | Interactive User     | Command ends with &, or Ctrl+Z pressed     | Job listed in jobs; can be moved with fg/bg      | FR-007, FR-008, FR-009 |
| UC-06 | Execute Built-in Command      | User / Script Runner | Built-in name entered                      | Shell state changed without fork                 | FR-009                 |
| UC-07 | Recall History / Use Aliases  | Interactive User     | Shell running                              | History listed; alias expanded before execution  | FR-010, FR-011         |
| UC-08 | Run Script Non-Interactively  | Script Runner        | Script file or piped stdin                 | Lines run in order; shell exits with last status | FR-012                 |

#### 4.2.1 Use Case Description – UC-03 Execute Multi-Stage Pipeline

| Field           | Description                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Main flow       | 1. User enters cmd1   cmd2   ...   cmdN. 2. Shell tokenizes, expands and parses the line into N commands. 3. Shell creates N-1 pipes and forks N children in one process group. 4. Each child connects its pipe ends with dup2, closes all other pipe descriptors, applies its redirections and calls execvp. 5. Shell closes its pipe descriptors and waits for all children. 6. \$? is set to the last stage's status and the prompt is shown. |
| Alternate flows | A1: a stage is a built-in → it runs in its child process (changes do not affect the shell, SEC-005). A2: a stage is not found → that stage exits 127; other stages still run.                                                                                                                                                                                                                                                                    |
| Exceptions      | E1:   with an empty side → syntax error, status 2, nothing executed. E2: pipe() or fork() fails → error message, already-started children are killed and reaped, status 1.                                                                                                                                                                                                                                                                       |

#### 4.3 Command Execution Activity Flow

1 Read line → 2 Add to history (interactive only) → 3 Tokenize → 4 Expand aliases, \$VAR, \$? → 5 Parse into Pipeline/Command structures → 6 If a single built-in without &: run in the shell process with temporary redirection, go to 9 → 7 Otherwise create pipes, fork each stage, apply dup2/redirections, execvp → 8 Foreground: wait for the process group; background: register a job and return immediately → 9 Update \$? → 10 Reap finished background jobs and report them → 11 Show prompt.

#### 4.4 Job Lifecycle State Model

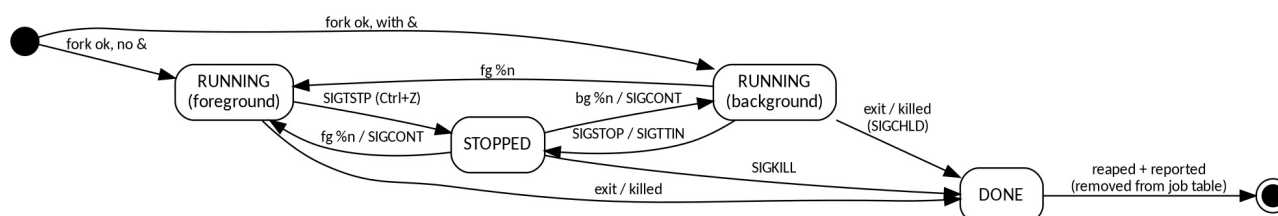


Figure 4.3 – Job state machine (states match Appendix B: RUNNING, STOPPED, DONE)

#### 4.5 Lexer Token Recognition Model

The tokenizer (FR-002) is a single-pass scanner with states START, WORD, IN\_SQUOTE, IN\_DQUOTE, ESCAPE and OPERATOR. A token boundary is emitted only outside quotes and escapes. The full state diagram is given in the SAD, Section 9.2.

## 5. System Features

Requirement IDs FR-001 to FR-010 keep the same numbers as in v1.0; their wording has been made measurable, and FR-010 has been narrowed to history only and FR-011 (aliases) and FR-012 (non-interactive mode) have been added. Each requirement lists priority, owner and verification method.

### 5.1 Interactive REPL & Dynamic Prompt

**Description and Priority:** The shell repeatedly shows a prompt, reads one line, executes it and shows the prompt again. Priority — High.

**Stimulus/Response Sequences:**

- User launches the binary from a terminal → prompt `user@host: cwd$` appears.
- User enters a line and presses Enter → line is executed; prompt is shown again with the updated directory.
- User presses Ctrl+D on an empty line → shell saves history and exits with the last status.

| ID     | Functional Requirement                                                                                                                                                                                                                                         | Priority | Owner  | Verification           |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|------------------------|
| FR-001 | In interactive mode the system shall display the prompt <code>user@hostname: cwd\$</code> (home directory shown as <code>~</code> ) before reading each input line, and shall accept input lines of any length up to available memory (verified up to 64 KiB). | High     | Anshul | System test ST-REPL-01 |

### 5.2 Lexical Analysis & Expansion

**Description and Priority:** Input is split into words and operators honouring quotes and escapes, then `$VAR` and `$?` are substituted. Priority — High.

**Stimulus/Response Sequences:**

- User enters a line with quotes, escapes or `$` tokens → tokens are produced before parsing.
- Line contains an unterminated quote → syntax error is reported and nothing runs.

| ID     | Functional Requirement                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | Priority | Owner  | Verification       |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|--------------------|
| FR-002 | The system shall split each input line into word and operator tokens ( <code> </code> , <code>&lt;</code> , <code>&gt;</code> , <code>&gt;&gt;</code> , <code>2&gt;</code> , <code>&amp;&gt;</code> , <code>&amp;</code> ) such that: text in single quotes is literal; text in double quotes keeps spaces and allows <code>\$</code> expansion; outside quotes a backslash makes the next character literal; inside double quotes a backslash escapes only <code>\$</code> , <code>"</code> and <code>\</code> . An unterminated quote shall produce a syntax error with exit status 2 and no command executed. | High     | Anshul | Unit UT-LEX-01..03 |
| FR-003 | The system shall replace <code>\$NAME</code> with the value of environment variable <code>NAME</code> (empty string if unset) and <code>\$?</code> with the exit status of the previous foreground command, except inside single quotes. Expanded values are not split into multiple words (a deliberate deviation from bash: after <code>X='a b', ls \$X</code> passes one argument <code>a b</code> ).                                                                                                                                                                                                         | High     | Anshul | Unit UT-EXP-01..02 |

### 5.3 External Command Execution

**Description and Priority:** Commands that are not built-ins are located on `PATH` and run in a child process. Priority — High.

**Stimulus/Response Sequences:**

- User enters `ls -l` → child runs `/bin/ls`; shell waits; `$?` = `ls` status.
- User enters an unknown command → error message, `$?` = 127.

| ID     | Functional Requirement                                                                                                                                                                                                                                                                                                                                                                                                                          | Priority | Owner   | Verification              |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|---------|---------------------------|
| FR-004 | For a command that is not a built-in, the system shall search the directories in PATH in order (or use the given path if the name contains /), run it in a child process with fork and execvp, wait for it with waitpid and set \$? to its exit status (128+n if killed by signal n). If not found it shall print <code>simple_shell: &lt;name&gt;: command not found</code> and set \$? to 127; if found but not executable, \$? shall be 126. | High     | Archana | Integration IT-EXE-01..02 |

## 5.4 Multi-Stage Pipelines

**Description and Priority:** Commands joined by | run concurrently with each stage's stdout connected to the next stage's stdin. Priority — High.

**Stimulus/Response Sequences:**

- User enters `cmd1 | cmd2 | cmd3` → three children run in one process group; output of the last stage is shown.

| ID     | Functional Requirement                                                                                                                                                                                                                                                                       | Priority | Owner   | Verification                          |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|---------|---------------------------------------|
| FR-005 | The system shall execute pipelines of 2 to at least 16 stages, connecting the stdout of stage i to the stdin of stage i+1 with POSIX pipes, running all stages in one process group, closing every unused pipe descriptor in every process, and setting \$? to the status of the last stage. | High     | Archana | Unit UT-PAR-01; Integration IT-PIP-01 |

## 5.5 File Stream Redirection

**Description and Priority:** Standard streams of a command are connected to files. Priority — High.

**Stimulus/Response Sequences:**

- User enters `cmd > out` → out is created/truncated with cmd's stdout.
- Target cannot be opened → error, \$? = 1, command not run.

| ID     | Functional Requirement                                                                                                                                                                                                                                                                                                                                                                                                                                         | Priority | Owner   | Verification              |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|---------|---------------------------|
| FR-006 | The system shall support <code>&lt; file</code> (stdin from file), <code>&gt; file</code> (stdout, create/truncate), <code>&gt;&gt; file</code> (stdout, create/append), <code>2&gt; file</code> (stderr, create/truncate) and <code>&amp;&gt; file</code> (stdout and stderr, create/truncate), creating new files with mode 0666 masked by the umask. If a file cannot be opened, the system shall print an error, set \$? to 1 and not execute the command. | High     | Ashlesh | Integration IT-RED-01..02 |

## 5.6 Job Control & Signal Handling

**Description and Priority:** Background execution, job table, and forwarding of terminal signals to the foreground job. Priority — High.

**Stimulus/Response Sequences:**

- User enters `sleep 60 &` → [1] <pid> printed; prompt returns immediately.
- User presses Ctrl+C / Ctrl+Z → foreground job is interrupted / stopped; shell keeps running.

| ID     | Functional Requirement                                                                                                                                                                                                                                                         | Priority | Owner   | Verification          |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|---------|-----------------------|
| FR-007 | The system shall run a command line ending in & in the background without waiting, print [job_id] pid, assign job IDs starting at 1 (reusing the lowest free ID), and print [job_id]+ Done <command> before the next prompt after the job finishes.                            | High     | Ashlesh | Integration IT-JOB-01 |
| FR-008 | In interactive mode the system shall deliver SIGINT (Ctrl+C) and SIGTSTP (Ctrl+Z) only to the foreground process group, shall itself not terminate or stop on these signals, and shall record a stopped job in the job table as STOPPED and print [job_id]+ Stopped <command>. | High     | Ashlesh | System ST-SIG-01..02  |

## 5.7 Built-in Commands

**Description and Priority:** Commands that change or report shell state run inside the shell process. Priority —

High.

#### Stimulus/Response Sequences:

- User enters `cd /tmp` → shell's working directory changes; next prompt shows `/tmp`.

| ID     | Functional Requirement                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Priority | Owner                                                                                                                                                                                                                                                                                                | Verification                           |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| FR-009 | The system shall execute the built-ins <code>cd</code> (no argument or <code>~</code> → <code>\$HOME</code> , <code>~/path</code> , <code>-</code> → previous directory), <code>pwd</code> , <code>echo</code> (with <code>-n</code> ), <code>export</code> , <code>unset</code> , <code>env</code> , <code>exit</code> [ <code>n</code> ], <code>jobs</code> , <code>fg</code> [ <code>%n</code> ], <code>bg</code> [ <code>%n</code> ], <code>history</code> , <code>alias</code> and <code>unalias</code> in the shell process without fork when they are a single foreground command (inside a pipeline or with <code>&amp;</code> they run in a child process), and shall return 0 on success and a non-zero status with an error message on failure. | High     | Atishay ( <code>cd</code> , <code>pwd</code> , <code>echo</code> , <code>exit</code> , <code>history</code> , <code>alias</code> , <code>unalias</code> ); Anshul ( <code>export</code> , <code>unset</code> , <code>env</code> ); Ashlesh ( <code>jobs</code> , <code>fg</code> , <code>bg</code> ) | Unit UT-BI-01;<br>Integration IT-BI-02 |

## 5.8 Session History

**Description and Priority:** Executed command lines are remembered across sessions. Priority — Medium.

#### Stimulus/Response Sequences:

- User enters commands → they appear in history.
- User exits and relaunches → previous commands are still listed.

| ID     | Functional Requirement                                                                                                                                                                                                                                               | Priority | Owner   | Verification                             |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|---------|------------------------------------------|
| FR-010 | In interactive mode the system shall append each non-empty input line to an in-memory history of 1,000 entries (oldest dropped first), list it numbered with <code>history</code> , save it to <code>~/.simple_shell_history</code> on exit and load it at start-up. | Medium   | Atishay | Unit UT-HIS-01;<br>Integration IT-HIS-02 |

## 5.9 Aliases

**Description and Priority:** Short names for longer command lines. Priority — Medium.

#### Stimulus/Response Sequences:

- User enters `alias ll='ls -l'` then `ll /` → `ls -l /` runs.

| ID     | Functional Requirement                                                                                                                                                                                                                                                                           | Priority | Owner   | Verification   |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|---------|----------------|
| FR-011 | The system shall define aliases with <code>alias name='value'</code> , list them with <code>alias</code> , remove them with <code>unalias name</code> , and replace the first word of a command with its alias value before parsing; an alias that refers to itself shall be expanded only once. | Medium   | Atishay | Unit UT-ALI-01 |

## 5.10 Non-Interactive (Script) Mode

**Description and Priority:** Running a file of commands for automation. Priority — Medium.

#### Stimulus/Response Sequences:

- Runner executes `simple_shell script.sh` or `cat script.sh | simple_shell` → each line runs in order with no prompt.

| ID     | Functional Requirement                                                                                                                                                                                                                     | Priority | Owner   | Verification     |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|---------|------------------|
| FR-012 | When given a script file argument or when stdin is not a terminal, the system shall execute input lines in order without printing a prompt, disable job control and history saving, and exit with the status of the last executed command. | Medium   | Archana | System ST-BAT-01 |

## 6. Other Nonfunctional Requirements

### 6.1 Performance Requirements

| ID      | Category    | Measurable Requirement                                                                                                                                                                                                                                                                                                                  | Verification    |
|---------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| NFR-001 | Performance | Mean dispatch overhead per simple external command shall be $\leq 2.0$ ms, measured as (time to run a 1,000-line script of <code>/bin/true</code> through the shell – time to fork/exec <code>/bin/true</code> 1,000 times from a C baseline harness) $\div$ 1,000, in the reference Docker container (Ubuntu 22.04, 2 vCPU, 4 GB RAM). | Benchmark PT-01 |
| NFR-008 | Performance | With a full 1,000-entry history file, the first prompt shall appear within 100 ms of process start (median of 20 runs).                                                                                                                                                                                                                 | Benchmark PT-02 |

### 6.2 Safety Requirements

Safety here means protecting the host and the user's session from the shell's own defects.

| ID      | Category        | Measurable Requirement                                                                                                                                                                                                                                                   | Verification          |
|---------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| NFR-002 | Memory safety   | Running the 1,000-command regression script under <code>valgrind --leak-check=full --error-exitcode=1</code> shall report 0 bytes definitely or indirectly lost and 0 memory errors.                                                                                     | Valgrind MEM-01       |
| NFR-003 | Process hygiene | After 50 background jobs complete, no child of the shell shall remain in zombie (Z) state for more than 100 ms after it exits.                                                                                                                                           | ZMB-01 (ps sampling)  |
| NFR-004 | Robustness      | The shell process shall survive 100% of the negative test suite ( $\geq 30$ inputs: malformed quotes, missing binaries, permission-denied files, empty pipeline sides, 100 KB lines, interrupted system calls), printing an error and showing the next prompt each time. | Negative suite NEG-01 |

### 6.3 Security Requirements

#### 6.3.1 Security Objectives

| ID   | Security Objective                                                                                                                                                                                        | Realised by               |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| SO-1 | <b>Integrity of the shell and host:</b> no input line, file content or environment value shall be able to corrupt shell memory, inject extra commands or cause code execution beyond what the user typed. | SEC-001, SEC-002, SEC-005 |
| SO-2 | <b>Confidentiality of user data:</b> command history and redirected files shall be readable only by the parties the file-system permissions allow; no shell-held descriptor shall leak to child programs. | SEC-003, SEC-006, SEC-007 |
| SO-3 | <b>Least privilege:</b> the shell shall never obtain, keep or exercise privileges beyond those of the invoking user.                                                                                      | SEC-003, SEC-004          |

#### 6.3.2 Security Requirements

| ID      | Security Requirement                                                                                                                                                                                                                                                                                                                                               | Objective  | Validation              |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|-------------------------|
| SEC-001 | All input and token buffers shall be dynamically sized or bounds-checked; an AddressSanitizer build shall report 0 errors over a fuzz corpus of $\geq 1,000$ random lines of up to 1 MB.                                                                                                                                                                           | SO-1       | ASan fuzz SEC-T01       |
| SEC-002 | Arguments shall be passed to <code>execvp</code> as an <code>argv</code> array exactly as tokenized; characters such as <code>;</code> , backtick and <code>\$ ( )</code> inside quotes shall reach the program as literal bytes; the source shall contain no calls to <code>system()</code> , <code>popen()</code> or <code>exec</code> of <code>/bin/sh</code> . | SO-1       | SEC-T02 + source grep   |
| SEC-003 | Redirection files shall be opened with the invoking user's credentials only; a permission-denied open shall fail with <code>\$? = 1</code> and leave the target file unchanged.                                                                                                                                                                                    | SO-2, SO-3 | SEC-T03                 |
| SEC-004 | The shell shall refuse to start (message, exit status 1) if its real and effective user or group IDs differ, so it cannot run with <code>setuid/setgid</code> privileges.                                                                                                                                                                                          | SO-3       | SEC-T04                 |
| SEC-005 | <code>export</code> , <code>unset</code> or <code>cd</code> executed inside a pipeline stage or background job shall not change the shell's own environment or working directory.                                                                                                                                                                                  | SO-1       | SEC-T05                 |
| SEC-006 | The history file shall be created with mode 0600; if an existing file has wider permissions, the shell shall change it to 0600 at load and print a warning.                                                                                                                                                                                                        | SO-2       | SEC-T06 (stat -c %a)    |
| SEC-007 | Files opened by the shell for its own use (history file) shall have <code>FD_CLOEXEC</code> set so no descriptor other than those for 0, 1, 2 and requested redirections is inherited by child programs.                                                                                                                                                           | SO-2       | SEC-T07 (/proc/self/fd) |

## 6.4 Software Quality Attributes

| ID      | Category        | Measurable Requirement                                                                                                                                                                                                                                                                                                                                              | Verification          |
|---------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| NFR-005 | Maintainability | Lexer, expander, parser, executor, redirection, job control, built-ins and history/alias shall each be in separate <code>.c/.h</code> files; shared state shall be passed via a <code>ShellContext</code> structure with 0 non-static global variables outside <code>signals.c</code> (checked with <code>nm</code> ); <code>Cppcheck</code> shall report 0 errors. | QA-01 (nm + Cppcheck) |
| NFR-006 | Portability     | The source shall compile with 0 warnings under <code>gcc -std=c11 -Wall -Wextra -Werror -pedantic</code> and pass 100% of the integration suite on both the Ubuntu 22.04 and Debian 12 Docker images.                                                                                                                                                               | PORT-01 (CI matrix)   |
| NFR-007 | Usability       | 100% of shell-generated error messages in the negative suite shall be written to <code>stderr</code> and begin with <code>simple_shell: .</code>                                                                                                                                                                                                                    | NEG-01                |

## 6.5 Business Rules

- BR-1: Built-ins that change shell state (`cd`, `export`, `unset`, `exit`, `alias`, `unalias`, `fg`, `bg`) take effect only when run as a single foreground command in the shell process; inside a pipeline or with a trailing `&` they run in a child process and do not affect the shell.
- BR-2: In a pipeline, every stage runs in its own child process, and every unused pipe descriptor is closed in every process.
- BR-3: `SIGCHLD` only sets a flag; reaping is done with `waitpid(-1, &status, WNOHANG | WUNTRACED)` in the main loop.
- BR-4: If any redirection of a command fails, the command is not executed and `$?` is 1.
- BR-5: An empty or whitespace-only line re-displays the prompt without changing `$?` or history.
- BR-6: Job IDs are local to a session and are not the same as process IDs.

## 7. Other Requirements

- **Portability:** core functionality uses only POSIX.1-2008 interfaces.
- **Packaging:** the shell builds and runs in Docker (reproducible grading/demo) and on bare metal/VM.

- **Internationalisation:** out of scope; single-byte English locale assumed, UTF-8 bytes passed through unchanged but not validated.
- **Licensing/reuse:** original academic code; test frameworks and sanitizers are build/QA dependencies only.
- **Database:** not applicable; the only persistent data is the flat history file (FR-010).

## Appendix A: Glossary

| Term                 | Definition                                                                                          |
|----------------------|-----------------------------------------------------------------------------------------------------|
| REPL                 | Read-Eval-Print Loop: read a line, execute it, show results, repeat.                                |
| PID / PGID           | Process ID / Process Group ID; a pipeline's processes share one PGID for signal delivery.           |
| fork / exec          | fork() creates a copy of the calling process; execvp() replaces a process image with a new program. |
| Pipe                 | Unidirectional kernel buffer connecting one process's output to another's input.                    |
| File descriptor (fd) | Small integer handle for an open file, pipe end or terminal.                                        |
| dup2                 | Copies one descriptor onto another number; used to rewire stdin/stdout/stderr.                      |
| Signal               | Asynchronous kernel notification (SIGINT, SIGTSTP, SIGCHLD, SIGCONT).                               |
| Zombie               | Terminated child whose status has not yet been collected with waitpid().                            |
| TTY / pty            | (Pseudo-)terminal device connecting keyboard and screen to the shell.                               |
| Job                  | A pipeline tracked by the shell with a job ID and state RUNNING, STOPPED or DONE.                   |
| Built-in             | A command executed inside the shell process rather than as a separate program.                      |
| WNOHANG              | waitpid() flag that returns immediately if no child has changed state.                              |
| Alias                | User-defined name replaced by a command string before parsing.                                      |
| FD_CLOEXEC           | Descriptor flag that closes the descriptor automatically on exec.                                   |

## Appendix B: Field Layouts

Simple Shell has no database; the equivalent field layouts are the core in-memory structures. Full C definitions are in the SAD, Section 9.1.

| Structure    | Field         | Data Type      | Description                                                | Mandatory |
|--------------|---------------|----------------|------------------------------------------------------------|-----------|
| Token        | type          | enum TokenType | WORD, PIPE, LESS, GREAT, DGREAT, ERR_GREAT, AND_GREAT, AMP | Y         |
| Token        | text          | char*          | Token text after quote removal                             | Y (WORD)  |
| Command      | argv          | char**         | NULL-terminated argument vector for execvp                 | Y         |
| Command      | argc          | int            | Number of arguments                                        | Y         |
| Command      | input_file    | char*          | Path for < redirection                                     | N         |
| Command      | output_file   | char*          | Path for >, >> or &> redirection                           | N         |
| Command      | append_output | int            | 1 = append (>>), 0 = truncate                              | Y         |
| Command      | error_file    | char*          | Path for 2> redirection (same as output_file for &>)       | N         |
| Pipeline     | commands      | Command*       | Array of commands in pipeline order                        | Y         |
| Pipeline     | command_count | int            | Number of stages (1-16 verified)                           | Y         |
| Pipeline     | is_background | int            | 1 if the line ends with &                                  | Y         |
| Pipeline     | text          | char*          | Original command text for job messages                     | Y         |
| Job          | job_id        | int            | Session-local job number ( $\geq 1$ )                      | Y         |
| Job          | pgid          | pid_t          | Process group of the job                                   | Y         |
| Job          | state         | enum JobState  | RUNNING, STOPPED or DONE                                   | Y         |
| Job          | is_foreground | int            | 1 while the job owns the terminal                          | Y         |
| HistoryEntry | line          | char*          | Command text (max 1,000 entries, ring buffer)              | Y         |
| Alias        | name / value  | char* / char*  | Alias name and replacement text                            | Y         |

## Appendix C: Requirement Traceability Matrix

Architecture references (C-xx) and design references point to the SAD; test IDs point to the Test Plan. Code file references are the planned file names from the SAD module design and will be confirmed during implementation.

| Req. ID | Brief Description          | Architecture Ref. | Design Ref.                  | Code File Ref.               | Test Case ID             | System Test ID        |
|---------|----------------------------|-------------------|------------------------------|------------------------------|--------------------------|-----------------------|
| FR-001  | Prompt & line input        | C-01              | SAD 9.3 REPL loop            | src/repl.c                   | —                        | ST-REPL-01, ST-E2E-01 |
| FR-002  | Tokenization               | C-02              | SAD 9.2 Lexer DFA            | src/lexer.c                  | UT-LEX-01..03            | ST-E2E-01             |
| FR-003  | \$VAR / \$? expansion      | C-03, C-10        | SAD 10.2 expander API        | src/expander.c, src/env.c    | UT-EXP-01..02            | ST-E2E-01             |
| FR-004  | External command execution | C-05              | SAD 10.2 executor API; SD-01 | src/executor.c               | IT-EXE-01..02            | ST-E2E-01             |
| FR-005  | Pipelines                  | C-04, C-05        | SD-01                        | src/parser.c, src/executor.c | UT-PAR-01, IT-PIP-01     | ST-E2E-01             |
| FR-006  | Redirection                | C-04, C-06        | SD-01; SAD 11                | src/parser.c, src/redirect.c | UT-PAR-01, IT-RED-01..02 | ST-E2E-01             |
| FR-007  | Background jobs            | C-07              | SD-02; Fig. 4.3              | src/jobs.c                   | IT-JOB-01                | ST-SIG-02             |
| FR-008  | Signal forwarding          | C-07              | SD-02                        | src/signals.c, src/jobs.c    | —                        | ST-SIG-01..02         |
| FR-009  | Built-ins                  | C-08              | SAD 10.3 built-in table      | src/builtins.c               | UT-BI-01, IT-BI-02       | ST-E2E-01, ST-SIG-02  |
| FR-010  | History                    | C-09              | SD-03                        | src/history.c                | UT-HIS-01, IT-HIS-02     | ST-E2E-01             |
| FR-011  | Aliases                    | C-09, C-03        | SD-03                        | src/alias.c                  | UT-ALI-01                | ST-E2E-01             |
| FR-012  | Script mode                | C-01              | SAD 9.3 REPL loop            | src/main.c, src/repl.c       | —                        | ST-BAT-01             |
| NFR-001 | Dispatch $\leq 2$ ms       | C-05              | SAD 8 tactics                | src/executor.c               | —                        | PT-01                 |
| NFR-002 | No leaks                   | All               | SAD 11 cleanup               | all                          | —                        | MEM-01                |
| NFR-003 | No zombies                 | C-07              | SD-02                        | src/jobs.c                   | —                        | ZMB-01                |
| NFR-004 | Robustness                 | All               | SAD 11 error handling        | all                          | UT-LEX-02                | NEG-01                |
| NFR-005 | Modularity                 | All               | SAD 5 components             | all                          | —                        | QA-01                 |
| NFR-006 | Portability                | —                 | SAD 6 deployment             | Makefile, Dockerfile         | —                        | PORT-01               |
| NFR-007 | Error format               | C-10              | SAD 11.2                     | src/error.c                  | IT-EXE-02                | NEG-01                |
| NFR-008 | Start-up $\leq 100$ ms     | C-01, C-09        | SD-03                        | src/history.c                | —                        | PT-02                 |
| SEC-001 | Bounds-checked buffers     | C-01, C-02        | SAD 7 security               | src/lexer.c, src/util.c      | —                        | SEC-T01               |
| SEC-002 | No injection               | C-02, C-05        | SAD 7 security               | src/lexer.c, src/executor.c  | —                        | SEC-T02               |
| SEC-003 | Respect file permissions   | C-06              | SAD 7 security               | src/redirect.c               | IT-RED-02                | SEC-T03               |
| SEC-004 | No setuid                  | C-01              | SD-03                        | src/main.c                   | —                        | SEC-T04               |
| SEC-005 | Environment isolation      | C-05, C-08        | SAD 7 security               | src/executor.c               | —                        | SEC-T05               |
| SEC-006 | History 0600               | C-09              | SD-03                        | src/history.c                | IT-HIS-02                | SEC-T06               |
| SEC-007 | No fd leakage              | C-09, C-05        | SAD 7 security               | src/history.c                | —                        | SEC-T07               |

**PES UNIVERSITY, BANGALORE**

Department of Computer Science and Engineering

JACKFRUIT MINI-PROJECT — UE24CS341A Software Engineering

# Software Architecture & Design Specification

for

**Simple Shell — POSIX-Compliant Command Line Interpreter in C**

| Item            | Details                                                                                                                             |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Project         | Simple Shell — POSIX-Compliant Command Line Interpreter in C                                                                        |
| Project ID      | SH-2026-10                                                                                                                          |
| Group           | Team 10 (G-10)                                                                                                                      |
| Team Members    | Anshul Poovaiah K – PES1UG24CS071 / Archana Shivakumar – PES1UG24CS078 / Ashlesh R V – PES1UG24CS088 / Atishay Jain – PES1UG24CS097 |
| Course          | UE24CS341A – Software Engineering, V Semester                                                                                       |
| Methodology     | Agile (Scrum, 6 sprints)                                                                                                            |
| Version         | 1.0                                                                                                                                 |
| Date            | 3 October 2026                                                                                                                      |
| Standards basis | IEEE Std 1016-2009 (SDD) and ISO/IEC/IEEE 42010 (architecture description)                                                          |

# Revision History

| Name    | Date        | Reason for Changes                                             | Version |
|---------|-------------|----------------------------------------------------------------|---------|
| Team 10 | 03-Oct-2026 | Initial architecture and design baseline derived from SRS v1.1 | 1.0     |

## Table of Contents

### Part A – Software Architecture

- 1. Introduction
- 2. Stakeholders and Concerns
- 3. Architectural Goals and Constraints
- 4. Architecture Pattern
- 5. Component View
- 6. Deployment View
- 7. Security Architecture
- 8. Quality Tactics and Traceability to Requirements

### Part B – Software Design

- 9. Detailed Design (data structures, lexer, control flow, UML sequence diagrams)
- 10. API Design
- 11. Error Handling
- 12. Design Decisions and Rationale
- 13. Work Division

# 1. Introduction

## 1.1 Purpose

This document describes the architecture and detailed design of Simple Shell (SH-2026-10). It shows how every requirement in SRS v1.1 is realised by a component, an interface and a design element, and it is the baseline for implementation and for the Test Plan.

## 1.2 Scope

Part A covers the high-level architecture: pattern, components, deployment and security architecture. Part B covers the software design: data structures, the lexer state machine, control flow, UML sequence diagrams, internal module APIs, the user-facing command interface and error handling.

## 1.3 Definitions

Terms such as REPL, PGID, pipe, dup2, job and built-in are defined in SRS Appendix A. C-xx identifies a component (Section 5); SD-xx identifies a sequence diagram (Section 9.4).

## 1.4 References

- Team 10 — Software Requirements Specification v1.1.
- Team 10 — Test Plan v1.0.
- IEEE Std 1016-2009; ISO/IEC/IEEE 42010:2011.
- POSIX.1-2008 (The Open Group Base Specifications Issue 7).

## 1.5 Overview

Sections 2–8 form the architecture description; Sections 9–13 form the design description. Requirement IDs (FR/NFR/SEC) are those of SRS v1.1.

# 2. Stakeholders and Concerns

| Stakeholder          | Concerns                                                                               | Addressed in       |
|----------------------|----------------------------------------------------------------------------------------|--------------------|
| Interactive User     | Correct, bash-like behaviour; responsive prompt; Ctrl+C/Ctrl+Z must not kill the shell | 5, 9.4 (SD-02), 11 |
| Script Runner        | Deterministic exit codes; no prompts in output                                         | 10.4, 11.2         |
| Developers (Team 10) | Clear module boundaries and stable internal APIs so four members can work in parallel  | 4, 5, 10, 13       |
| Evaluators / Faculty | Traceability, security, memory safety evidence                                         | 7, 8.2             |
| Maintainers          | Readable modules, no hidden global state                                               | 4.2, 12            |

# 3. Architectural Goals and Constraints

- **Correctness of process handling:** no zombies, no descriptor leaks, no deadlocked pipelines (FR-005, NFR-003).
- **Memory safety:** single ownership of every heap object, freed at a defined point (NFR-002, SEC-001).
- **Parallel development:** components owned by different members communicate only through the APIs in Section 10.
- **Low overhead:** no extra processes or interpreters between the user and fork/exec (NFR-001).
- **Constraints:** C11 + POSIX only; single-threaded design (signals are the only asynchronous events); signal

handlers limited to async-signal-safe calls; GitHub Actions CI on every PR.

## 4. Architecture Pattern

### 4.1 Chosen Pattern: Layered Architecture with a Pipe-and-Filter Command-Processing Core

The system is organised into four layers. Each layer uses only the layer below it and the shared support layer:

| Layer                                      | Components                                                                  | Responsibility                                                                                             |
|--------------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Interface layer                            | C-01 REPL & Prompt                                                          | Reads input (terminal or script), shows prompt, drives one command cycle                                   |
| Command-processing layer (Pipe-and-Filter) | C-02 Lexer → C-03 Expander → C-04 Parser                                    | Transforms a raw line into a Pipeline structure; each filter takes one data structure and returns the next |
| Execution layer                            | C-05 Executor, C-06 Redirection, C-07 Job Control & Signals, C-08 Built-ins | Creates processes, wires descriptors, manages jobs and signals, runs built-ins                             |
| State & support layer                      | C-09 History & Alias, C-10 Environment & Utilities                          | Holds session state; error reporting; safe allocation helpers                                              |

Inside the command-processing layer, data flows strictly one way: `char* line` → `TokenList` → expanded `TokenList` → `Pipeline`. Each filter can be unit-tested in isolation by feeding it its input structure, which is why this layer has the highest unit-test coverage.

### 4.2 Justification

- **Matches the problem:** a shell is naturally a sequence of transformations (lex → expand → parse → execute); Pipe-and-Filter makes each step replaceable and testable.
- **Supports parallel ownership:** the four members own different layers/filters, joined by the data structures in Section 9.1.
- **Isolates OS-specific code:** only the execution and support layers call `fork/exec/sigaction`, so lexer/parser tests run without creating processes.
- **Maintainability (NFR-005):** each component is one or two `.c` files with one header.

### 4.3 Alternatives Considered

| Alternative                                         | Reason rejected                                                                                             |
|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Monolithic single-file loop (typical student shell) | Untestable in isolation; violates NFR-005; blocks parallel work                                             |
| Full AST interpreter (bash-style grammar)           | Needed only for control structures, which are out of scope; adds complexity without benefit                 |
| Event-driven / multi-threaded design                | Signals + fork are unsafe with threads; a single-threaded loop with a self-pipe/flag is simpler and correct |

## 5. Component View

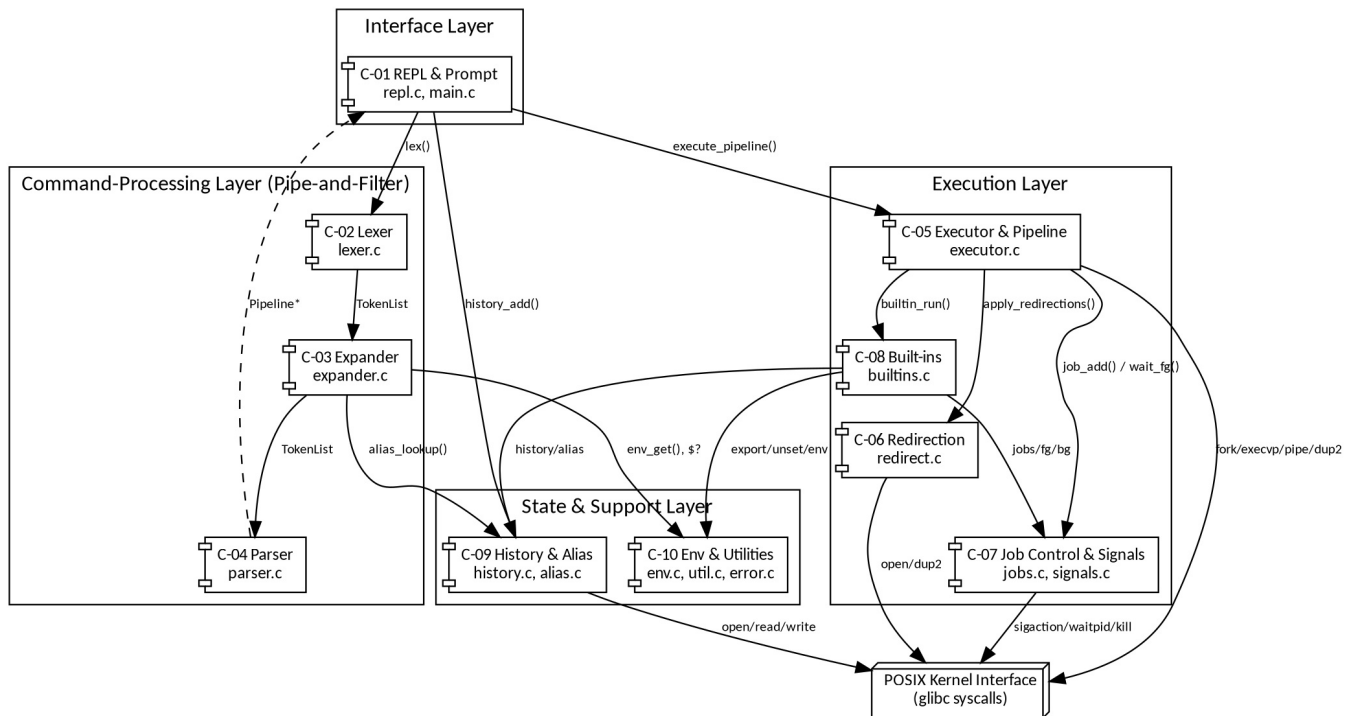


Figure 5.1 – UML component diagram (provided interfaces shown as labelled dependencies)

### 5.1 Component Descriptions

| ID   | Component               | Files                  | Responsibility                                                                               | Provided interface                                        | Requirements                     | Owner                       |
|------|-------------------------|------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------|----------------------------------|-----------------------------|
| C-01 | REPL & Prompt           | main.c, repl.c         | Start-up checks, mode detection (TTY or script), prompt rendering, read loop, shutdown       | repl_run(), prompt_render()                               | FR-001, FR-012, SEC-004, NFR-008 | Anshul                      |
| C-02 | Lexer                   | lexer.c, lexer.h       | Convert a line into tokens; quote and escape handling; syntax errors for unterminated quotes | lex(), token_list_free()                                  | FR-002, SEC-001, SEC-002         | Anshul                      |
| C-03 | Expander                | expander.c             | Alias substitution on first word; \$VAR and \$? expansion                                    | expand()                                                  | FR-003, FR-011                   | Anshul                      |
| C-04 | Parser                  | parser.c               | Group tokens into Commands and a Pipeline; attach redirections; detect trailing &            | parse(), pipeline_free()                                  | FR-005, FR-006, FR-007           | Archana                     |
| C-05 | Executor & Pipeline     | executor.c             | PATH resolution, fork/execvp, pipe wiring, foreground waiting, exit-status mapping           | execute_pipeline(), resolve_path()                        | FR-004, FR-005, NFR-001, SEC-005 | Archana                     |
| C-06 | Redirection             | redirect.c             | Open target files and dup2 them onto 0/1/2; save/restore for builtin-ins                     | apply_redirections(), save_std_fds(), restore_std_fds()   | FR-006, SEC-003                  | Ashlesh                     |
| C-07 | Job Control & Signals   | jobs.c, signals.c      | Job table, process groups, terminal ownership, SIGCHLD flag, reaping, fg/bg                  | job_add(), wait_foreground(), jobs_reap(), signals_init() | FR-007, FR-008, NFR-003          | Ashlesh                     |
| C-08 | Built-ins               | builtins.c             | Dispatch table and implementations of all built-ins                                          | builtin_lookup(), builtin_run()                           | FR-009                           | Atishay / Anshul / Ashlesh  |
| C-09 | History & Alias         | history.c, alias.c     | History ring buffer and file persistence; alias table                                        | history_add/load/save(), alias_set/get/remove()           | FR-010, FR-011, SEC-006, SEC-007 | Atishay                     |
| C-10 | Environment & Utilities | env.c, util.c, error.c | Environment access, checked allocation (xmalloc), error reporting                            | env_get/set/unset(), xmalloc(), sh_error()                | FR-003, NFR-007                  | Anshul (env), shared (util) |

### 5.2 Component Interaction Rules

- Components never read another component's internal data; all shared session state is in one ShellContext passed by pointer (NFR-005).
- Process-creation, descriptor-wiring and signal operations (fork, execvp, pipe, dup2, setpgid, tcsetpgrp, sigaction, waitpid, kill) are called only from C-05, C-06 and C-07; file I/O for history only from C-09. Simple state queries and changes made by other components are listed exceptions: getuid/geteuid/isatty in C-01, chdir/getcwd in C-08, getenv/setenv/unsetenv in C-10.
- Ownership: C-02 owns tokens until C-04 builds the Pipeline. execute\_pipeline() (C-05) takes ownership of the Pipeline: it frees it before returning for completed foreground commands and built-ins, or transfers it to a Job (C-07) for background or stopped jobs; C-07 frees it when the job is removed. The REPL never frees a Pipeline after passing it to execute\_pipeline().

## 6. Deployment View

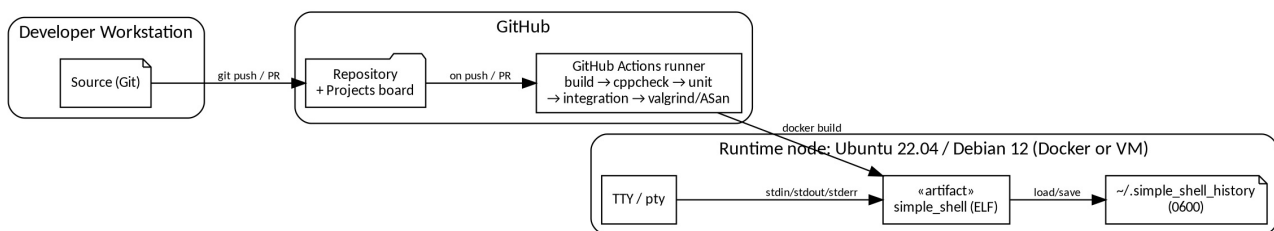


Figure 6.1 – Deployment and CI/CD view

The shell is a single ELF binary built by make. CI on GitHub Actions builds the Docker images for Ubuntu 22.04 and Debian 12 and runs, in order: compile with -Werror, Cppcheck/Clang-Tidy, unit tests, integration tests, Valgrind and the ASan fuzz job (NFR-006). At run time the binary needs only a terminal and the user's home directory for the history file.

## 7. Security Architecture

### 7.1 Trust Boundaries

| Boundary                   | Untrusted input crossing it                                | Entry component  |
|----------------------------|------------------------------------------------------------|------------------|
| TB-1 User / script → shell | Command lines of any length and content                    | C-01, C-02       |
| TB-2 Environment → shell   | Environment variable values (e.g., PATH, HOME)             | C-03, C-05, C-10 |
| TB-3 File system → shell   | History file contents and permissions; redirection targets | C-06, C-09       |
| TB-4 Shell → child process | Inherited descriptors, environment and privileges          | C-05             |

### 7.2 Threats and Controls

| Threat                                        | Control in design                                                                                                                                                                    | Requirement     |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| Buffer overflow from long or crafted input    | getline() with dynamic buffers; token buffers grow with a checked xrealloc; no fixed-size arrays for user data                                                                       | SEC-001         |
| Command injection through quotes or variables | No shell re-parsing: \$VAR and \$? values are never re-tokenized (alias values are the only re-lexed text, see 12); execvp receives an argv array; system()/popen() banned (CI grep) | SEC-002         |
| Privilege misuse / setuid execution           | At start-up compare getuid()/geteuid() and getgid()/getegid(); exit 1 if different                                                                                                   | SEC-004         |
| Writing files the user should not             | open() with the user's own credentials; no chmod/chown of targets; failure aborts the command                                                                                        | SEC-003         |
| Disclosure of history                         | history file created 0600; wider permissions corrected with fchmod at load                                                                                                           | SEC-006         |
| Descriptor leakage to children                | history file opened with O_CLOEXEC; pipe ends closed in every process after dup2                                                                                                     | SEC-007, FR-005 |
| Pipeline built-in corrupting shell state      | Built-ins inside pipelines run in the forked child only                                                                                                                              | SEC-005         |
| Signal-handler races                          | SIGCHLD handler only sets a volatile sig_atomic_t flag; reaping happens in the main loop                                                                                             | NFR-003         |

### 7.3 Security Validation Hooks

Every control above has a test in Test Plan Section 5.1. CI runs an ASan/UBSan build against the fuzz corpus and a grep check (the same check as SEC-T02) that fails the build if system(, popen( or an exec of /bin/sh appears in src/.

## 8. Quality Tactics and Traceability to Requirements

### 8.1 Quality Attribute Tactics

| Quality attribute | Tactic                                                                                                 | Requirement |
|-------------------|--------------------------------------------------------------------------------------------------------|-------------|
| Performance       | Single fork per stage; no extra shell process; PATH lookup with access(X_OK) and no directory scanning | NFR-001     |
| Start-up time     | History loaded with one buffered read; fixed-size ring buffer                                          | NFR-008     |
| Memory safety     | Single-owner rule; *_free() for every allocating API; Valgrind in CI                                   | NFR-002     |
| Reliability       | Every system call result checked; EINTR retried; shell never exits on a command error                  | NFR-004     |
| Process hygiene   | Non-blocking reap loop before each prompt and after each foreground wait                               | NFR-003     |
| Maintainability   | One component per concern; ShellContext instead of globals                                             | NFR-005     |

### 8.2 Requirements-to-Architecture Traceability

| Requirement   | Component(s) | Design element                   |
|---------------|--------------|----------------------------------|
| FR-001        | C-01         | 9.3 REPL loop                    |
| FR-002        | C-02         | 9.2 Lexer DFA                    |
| FR-003        | C-03, C-10   | 10.2 expand()                    |
| FR-004        | C-05         | 10.2 execute_pipeline(); SD-01   |
| FR-005        | C-04, C-05   | SD-01                            |
| FR-006        | C-04, C-06   | SD-01; 10.2 apply_redirections() |
| FR-007        | C-07         | SD-02; Fig 9.2                   |
| FR-008        | C-07         | SD-02; 9.3 signal dispositions   |
| FR-009        | C-08         | 10.3 built-in table              |
| FR-010        | C-09         | SD-03                            |
| FR-011        | C-09, C-03   | SD-03; 10.2 alias API            |
| FR-012        | C-01         | 9.3 REPL loop                    |
| NFR-001...008 | See 8.1      | 8.1 tactics                      |
| SEC-001...007 | See 7.2      | 7.2 controls                     |

## 9. Detailed Design

### 9.1 Data Structures

```
typedef enum { T_WORD, T_PIPE, T_LESS, T_GREAT, T_DGREAT,
               T_ERR_GREAT, T_AND_GREAT, T_AMP } TokenType;
typedef struct { TokenType type; char *text;
               unsigned char *qmask; /* per character: 0 unquoted,
                                     1 single-quoted, 2 double-quoted;
                                     lets '$HOME'$HOME expand only once */
} Token;
typedef struct { Token *items; size_t count, cap; } TokenList;

typedef struct {
    char **argv; int argc;
    char *input_file; /* < */
    char *output_file; /* > >> &> */
    int append_output; /* 1 for >> */
    char *error_file; /* 2> (or &>) */
} Command;

typedef struct {
    Command *commands; int command_count;
    int is_background; char *text; /* original line for job messages */
} Pipeline;

typedef struct { char **lines; int cap, start, count; } History; /* ring, cap = 1000 */
typedef struct { char *name; char *value; } Alias;
typedef struct { Alias *items; size_t count, cap; } AliasTable;

typedef enum { JOB_RUNNING, JOB_STOPPED, JOB_DONE } JobState;
typedef struct Job {
    int job_id; pid_t pgid; pid_t *pids; int npids;
    JobState state; int is_foreground; int last_status;
    char *text; struct Job *next;
} Job;

typedef struct {
    int interactive; /* isatty(STDIN_FILENO) && no script arg */
    int last_status; /* $? */
    pid_t shell_pgid; struct termios shell_tmodes;
    Job *jobs; History *history; AliasTable *aliases;
    char *prev_cwd; /* for cd - */
    int exit_requested; int exit_code;
} ShellContext;
```

## 9.2 Lexer State Machine

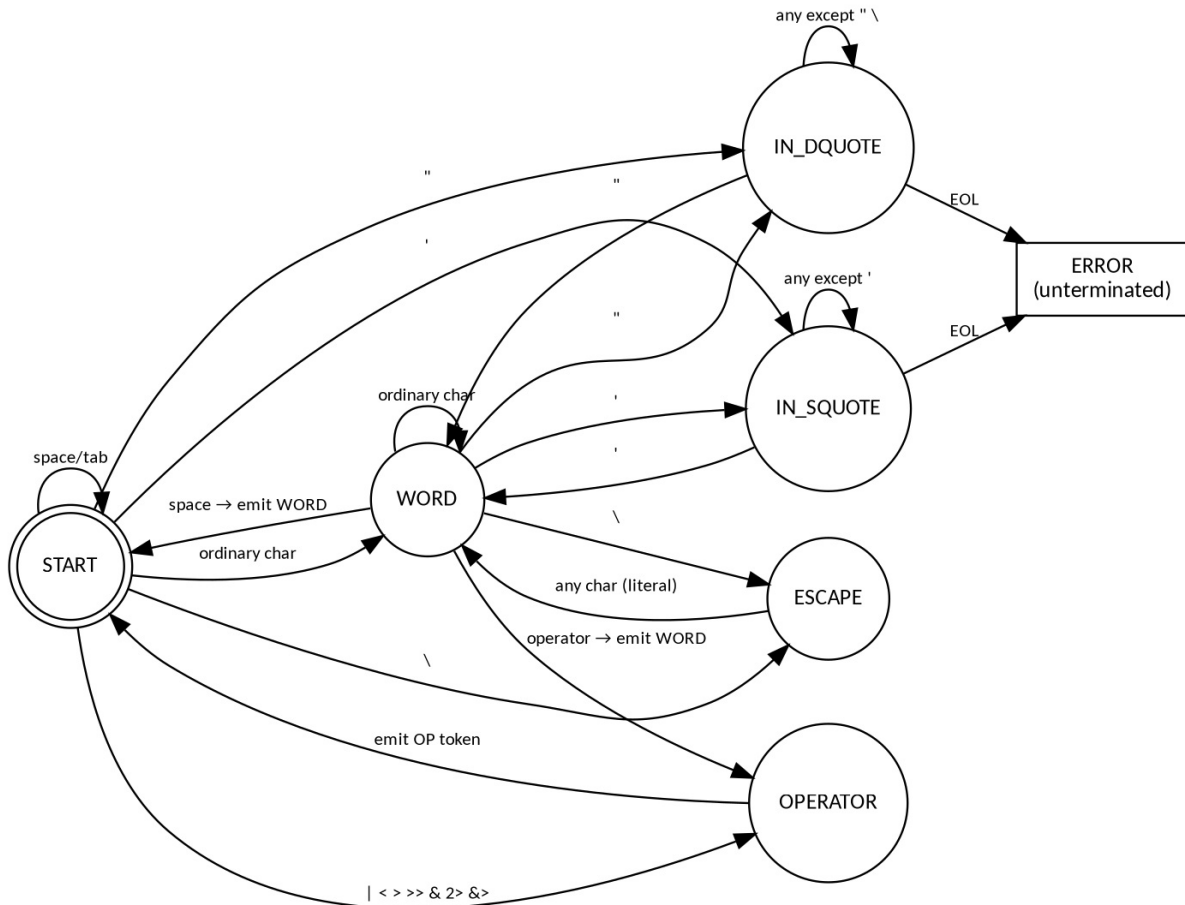


Figure 9.1 – Lexer DFA (inside IN\_DQUOTE, a backslash escapes only \$ " \; \$ is kept for the expander)

## 9.3 Main Control Flow

```

int repl_run(ShellContext *ctx, FILE *in) {
    char *line = NULL; size_t cap = 0;
    for (;;) {
        jobs_reap(ctx); /* report Done/Stopped jobs */
        if (ctx->interactive) prompt_render(ctx);
        ssize_t n = getline(&line, &cap, in); /* SEC-001: dynamic buffer */
        if (n < 0) { if (errno == EINTR) { clearerr(in); continue; } break; }
        if (is_blank(line)) continue; /* BR-5 */
        if (ctx->interactive) history_add(ctx->history, line);
        TokenList *tl = lex(line, &err); if (!tl) { report; ctx->last_status = 2;
continue; }
        if (expand(tl, ctx) != 0) { token_list_free(tl); continue; }
        Pipeline *pl = parse(tl, &err); token_list_free(tl);
        if (!pl) { report; ctx->last_status = 2; continue; }
        ctx->last_status = execute_pipeline(pl, ctx); /* takes ownership of pl */
        if (ctx->exit_requested) break;
    }
    free(line);
    if (ctx->interactive) history_save(ctx->history);
    return ctx->exit_requested ? ctx->exit_code : ctx->last_status;
}

```

**Signal dispositions (interactive mode):** the shell puts itself in its own process group, takes the terminal with `tcsetpgrp`, and ignores `SIGINT`, `SIGQUIT`, `SIGTSTP`, `SIGTTIN` and `SIGTTOU`. Children reset these to `SIG_DFL` before `execvp`, so `Ctrl+C/Ctrl+Z` affect only the foreground group (FR-008). `SIGCHLD` is handled with `SA_RESTART` and only sets a flag. In script mode (FR-012) job control is disabled and signals keep default dispositions.

# 9.4 UML Sequence Diagrams

SD-01: Execute a multi-stage pipeline with output redirection (UC-03, UC-04)

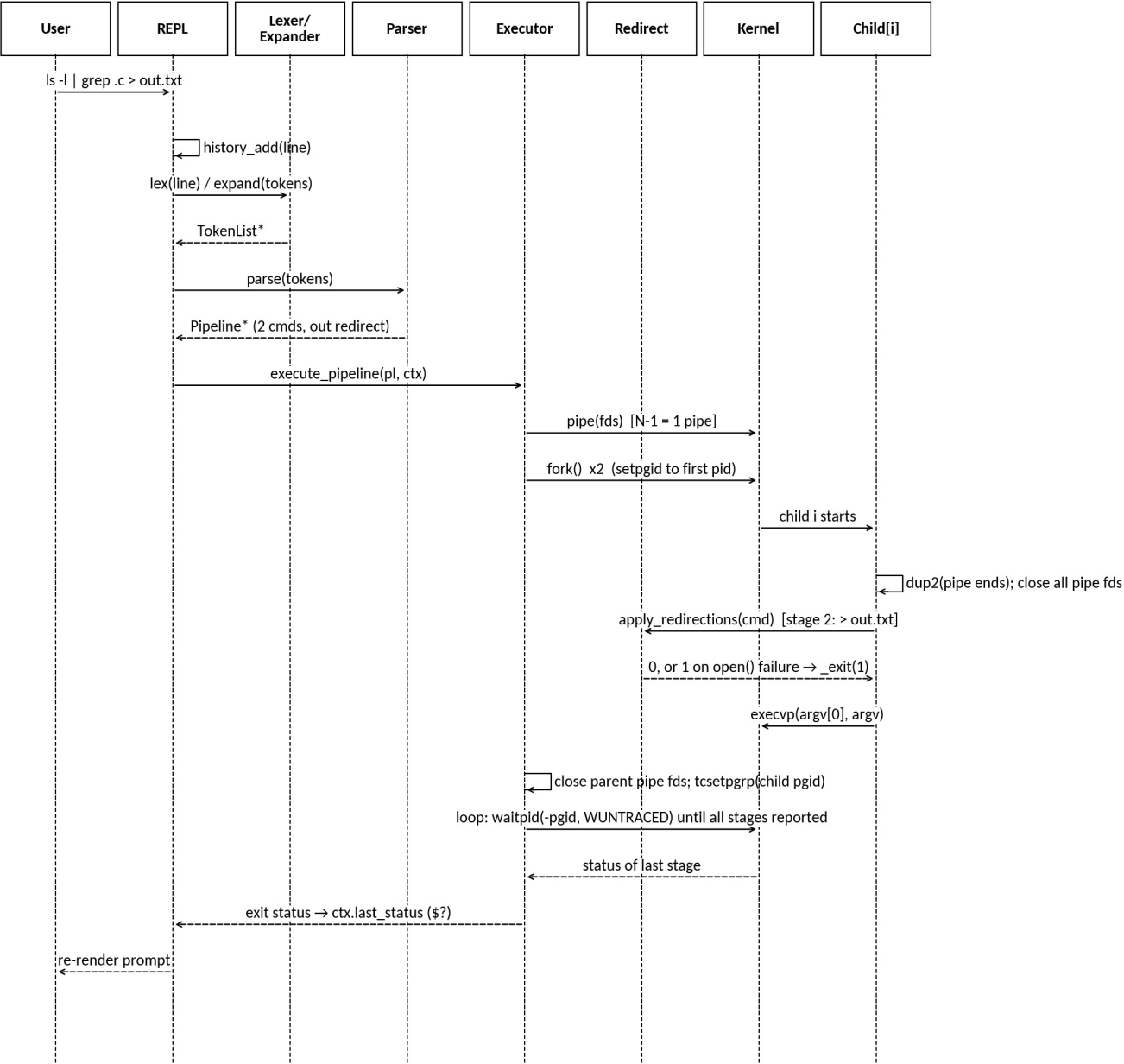


Figure 9.2 – SD-01: execute `ls -l | grep .c > out.txt` (FR-004, FR-005, FR-006)

## SD-02: Suspend a foreground job with Ctrl+Z, resume it with bg, reap on completion (UC-05)

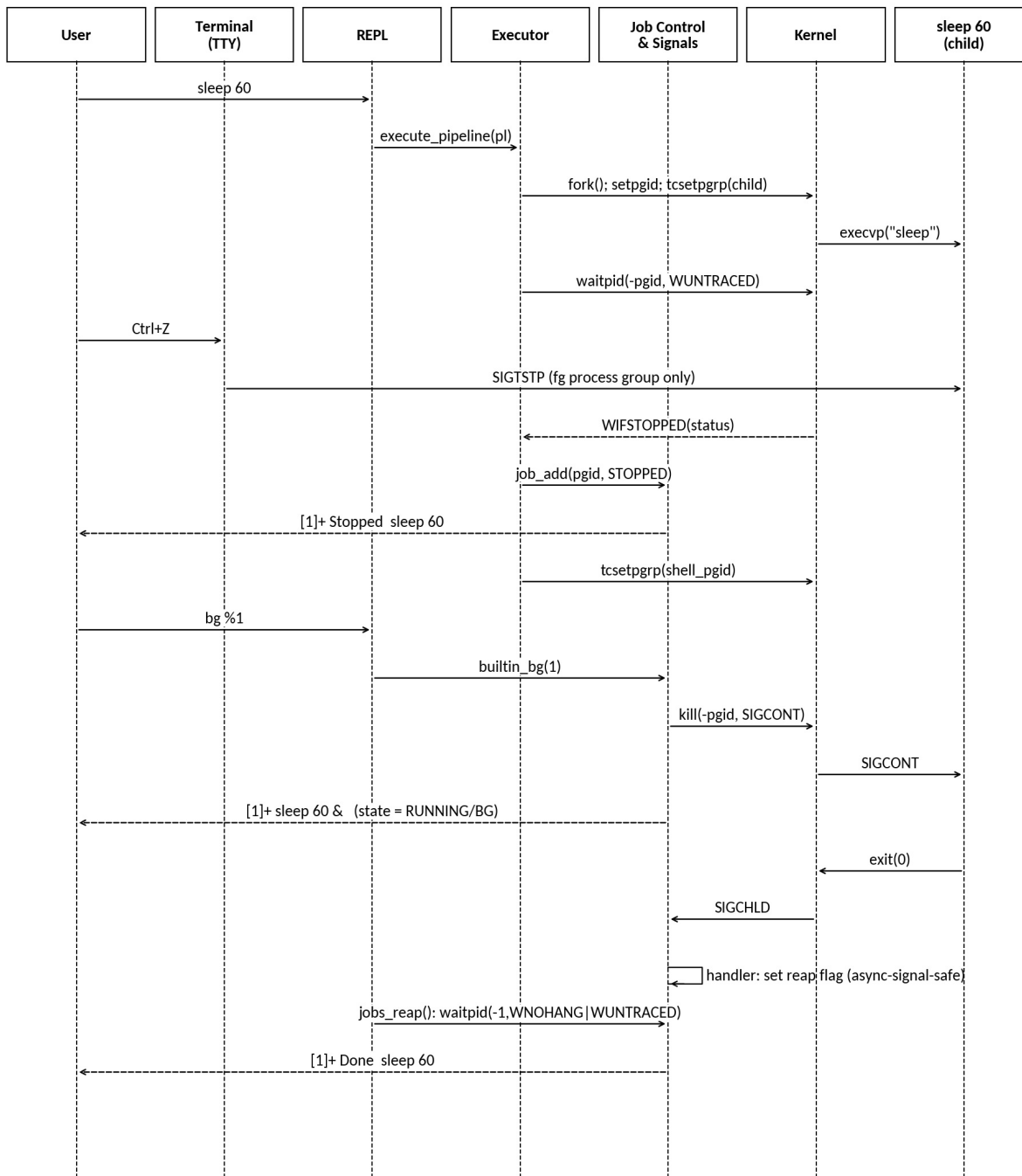


Figure 9.3 – SD-02: Ctrl+Z, bg and reaping of a job (FR-007, FR-008, FR-009, NFR-003)

### SD-03: Session start-up, alias definition and clean shutdown (UC-01, UC-06, UC-07)

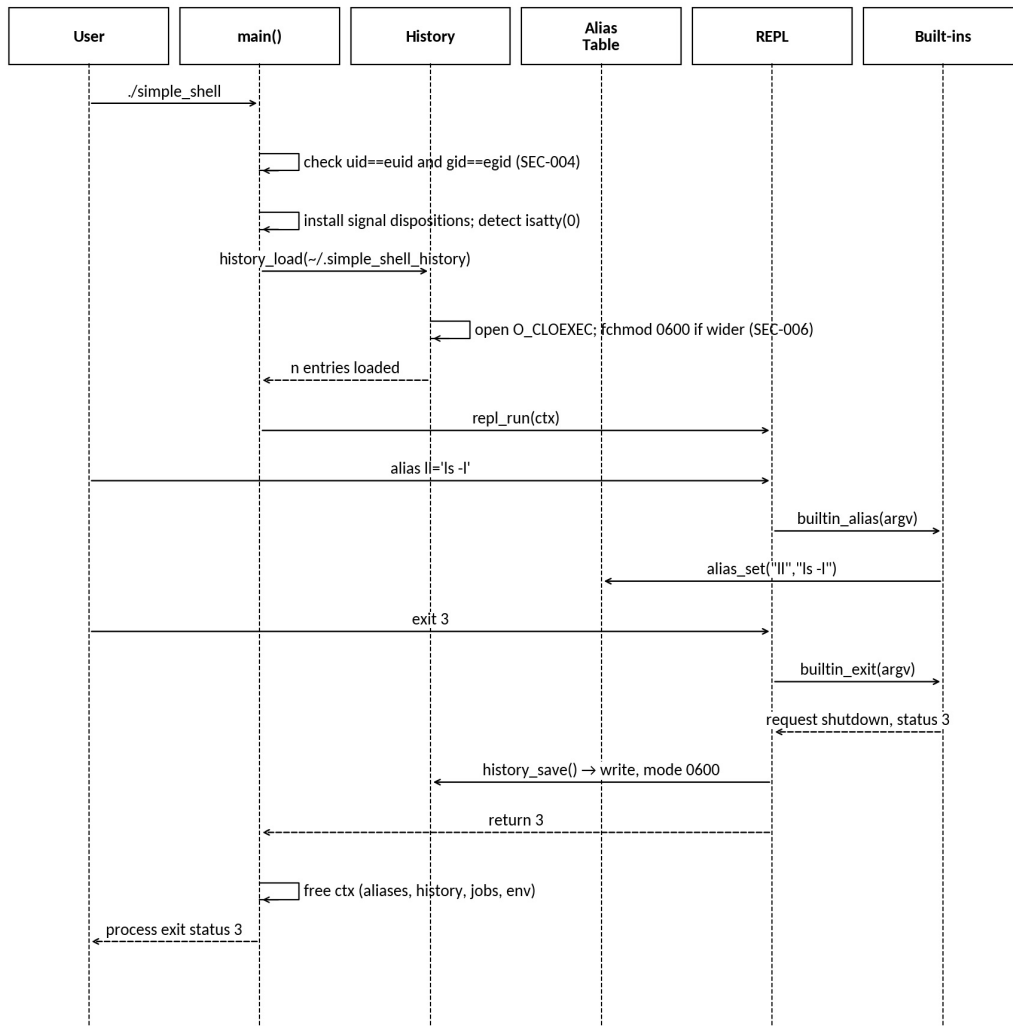


Figure 9.4 – SD-03: start-up, alias definition and shutdown (FR-010, FR-011, SEC-004, SEC-006)

## 9.5 Job State Machine

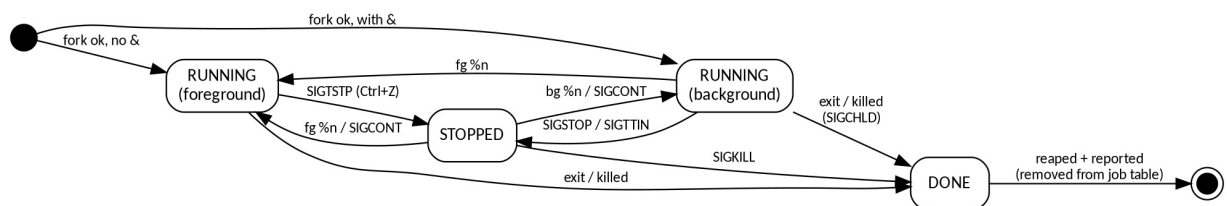


Figure 9.5 – Job state machine implemented in jobs.c

## 10. API Design

### 10.1 Conventions

- Functions returning `int` return 0 on success and a non-zero shell status on failure; functions returning pointers return NULL on failure and fill a `ShError` with code and message.
- Every API that allocates has a matching `*_free()`; the caller named in Section 5.2 owns the result.
- No API prints except `sh_error()` (C-10) and job notifications (C-07), so output format is centralised (NFR-007).
- Headers in `include/` are the frozen contract between owners; changes require a reviewed PR approved by the affected owner.

### 10.2 Internal Module APIs

| Component | Function                                                                                               | Parameters                            | Returns / Errors                                                                     | Pre / Post-conditions                                                                                                                                                                                     |
|-----------|--------------------------------------------------------------------------------------------------------|---------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| C-01      | <code>int repl_run(ShellContext *ctx, FILE *in)</code>                                                 | ctx initialised; in = stdin or script | Final exit status                                                                    | Post: history saved (interactive), all memory freed                                                                                                                                                       |
| C-01      | <code>void prompt_render(const ShellContext *ctx)</code>                                               | ctx                                   | —; falls back to \$ if user/host lookup fails                                        | Writes prompt to stdout and flushes                                                                                                                                                                       |
| C-02      | <code>TokenList *lex(const char *line, ShError *err)</code>                                            | NUL-terminated line                   | TokenList* or NULL; ERR_SYNTAX (unterminated quote), ERR_NOMEM                       | line not modified                                                                                                                                                                                         |
| C-02      | <code>void token_list_free(TokenList *tl)</code>                                                       | tl (may be NULL)                      | —                                                                                    | All token texts freed                                                                                                                                                                                     |
| C-03      | <code>int expand(TokenList *tl, ShellContext *ctx)</code>                                              | tokens, ctx (env, \$?, aliases)       | 0; ERR_NOMEM, ERR_SYNTAX (bad alias text)                                            | If the first word is an alias, its value is lexed with <code>lex()</code> and spliced in (once); then \$VAR/\$? are substituted using <code>qmask</code> ; substituted values are never re-lexed or split |
| C-04      | <code>Pipeline *parse(const TokenList *tl, ShError *err)</code>                                        | expanded tokens                       | Pipeline* or NULL; ERR_SYNTAX (empty pipe side, missing redirect target, & not last) | Each Command argv NULL-terminated                                                                                                                                                                         |
| C-04      | <code>void pipeline_free(Pipeline *pl)</code>                                                          | pl                                    | —                                                                                    | —                                                                                                                                                                                                         |
| C-05      | <code>int execute_pipeline(Pipeline *pl, ShellContext *ctx)</code>                                     | pl, ctx                               | Exit status of last stage; 1 on pipe/fork failure                                    | Takes ownership of pl: frees it on return, or transfers it to a Job (background/stopped)                                                                                                                  |
| C-05      | <code>char *resolve_path(const char *name, const char *path_env, int *err)</code>                      | command name, PATH value              | Malloc'd full path or NULL; err = 127 not found / 126 not executable                 | —                                                                                                                                                                                                         |
| C-06      | <code>int apply_redirections(const Command *c, ShError *err)</code>                                    | command                               | 0; 1 with ERR_OPEN (errno text)                                                      | Pre: called in child, or between save/restore in parent                                                                                                                                                   |
| C-06      | <code>int save_std_fds(int saved[3]) / void restore_std_fds(int saved[3])</code>                       | array of 3                            | 0 / —                                                                                | Used around parent-context built-ins                                                                                                                                                                      |
| C-07      | <code>void signals_init(ShellContext *ctx)</code>                                                      | ctx                                   | —                                                                                    | Dispositions per 9.3                                                                                                                                                                                      |
| C-07      | <code>Job *job_add(ShellContext *ctx, pid_t pgid, pid_t *pids, int n, Pipeline *pl, JobState s)</code> | job data                              | Job*; lowest free job_id                                                             | Prints [id] pid for background jobs                                                                                                                                                                       |
| C-07      | <code>int wait_foreground(ShellContext *ctx, Job *j)</code>                                            | job                                   | Status of last stage; job STOPPED on WIFSTOPPED                                      | Terminal returned to shell (tcsetpgrp)                                                                                                                                                                    |
| C-07      | <code>void jobs_reap(ShellContext *ctx)</code>                                                         | ctx                                   | —                                                                                    | <code>waitpid(-1, WNOHANG   WUNTRACED)</code> until 0; prints Done/Stopped                                                                                                                                |
| C-08      | <code>BuiltinFn builtin_lookup(const char *name)</code>                                                | argv[0]                               | Function pointer or NULL                                                             | —                                                                                                                                                                                                         |
| C-08      | <code>int builtin_run(BuiltinFn f, Command *c,</code>                                                  | built-in, command,                    | Built-in status                                                                      | Redirections applied and                                                                                                                                                                                  |

|      | ShellContext *ctx)                                                                                                 | ctx                   |                                                  | restored                                   |
|------|--------------------------------------------------------------------------------------------------------------------|-----------------------|--------------------------------------------------|--------------------------------------------|
| C-09 | int history_add / history_load / history_save (History *h, ...)                                                    | history, line or path | 0; ERR_IO (load/save failures are warnings only) | File mode 0600, O_CLOEXEC                  |
| C-09 | int alias_set(AliasTable*, const char *name, const char *value); const char *alias_get(...); int alias_remove(...) | table, name, value    | 0 / NULL / 1 if not found                        | Names validated: [A-Za-z0-9_]+             |
| C-10 | const char *env_get(const char*); int env_set(const char*, const char*); int env_unset(const char*)                | name, value           | Value or NULL; 0 / 1 on invalid name             | Wraps getenv/setenv/unsetenv               |
| C-10 | void sh_error(const char *ctx, const char *fmt, ...)                                                               | context, message      | —                                                | Writes simple_shell:<br>ctx: msg to stderr |

### 10.3 Built-in Command Interface

| Built-in        | Syntax                              | Success status  | Failure status / message                             | Owner   |
|-----------------|-------------------------------------|-----------------|------------------------------------------------------|---------|
| cd              | cd [dir   ~   ~/path   -]           | 0               | 1:simple_shell: cd: <dir>: No such file or directory | Atishay |
| pwd             | pwd                                 | 0               | 1 if getcwd fails                                    | Atishay |
| echo            | echo [-n] [args...]                 | 0               | —                                                    | Atishay |
| exit            | exit [n]                            | n (default \$?) | 2: numeric argument required                         | Atishay |
| export          | export NAME=value   NAME            | 0               | 1: not a valid identifier                            | Anshul  |
| unset           | unset NAME...                       | 0               | 1: not a valid identifier                            | Anshul  |
| env             | env                                 | 0               | —                                                    | Anshul  |
| jobs            | jobs                                | 0               | —                                                    | Ashlesh |
| fg / bg         | fg [%n] / bg [%n]                   | job status / 0  | 1: no such job                                       | Ashlesh |
| history         | history                             | 0               | —                                                    | Atishay |
| alias / unalias | alias [name='value'] / unalias name | 0               | 1: not found / invalid name                          | Atishay |

### 10.4 User-Facing Command Interface (external contract)

Invocation: simple\_shell (interactive if stdin is a TTY) or simple\_shell script.sh (script mode). Grammar supported: pipeline := command ( '|' command ) \* [ '&' ]; command := WORD+ ( redirect ) \*; redirect := ('<' | '>' | '>>' | '2>' | '&>') WORD. Exit status conventions are those of SRS Section 3.1. This contract is versioned with the release tag and will not change silently once other modules or tests depend on it.

## 11. Error Handling

### 11.1 Strategy

- **Never terminate the shell on a command error** (NFR-004): errors are converted into an exit status and a message, then the loop continues.
- **Check every system call**; on EINTR, retry read/waitpid/open; other errors are reported with strerror(errno).
- **Fail before side effects**: syntax errors are detected in lex/parse before any process is created; redirection errors are detected before execvp.
- **Errors in children** are reported by the child to stderr and returned through \_exit(code), never by returning into the parent's loop.
- **Out-of-memory**: xmalloc reports and returns NULL to the caller in the parent (the command is abandoned); in a child it calls \_exit(1).

### 11.2 Message Format and Status Codes

Format: `simple_shell: <context>: <message>` on stderr (NFR-007). Status codes: 0 success, 1 general/redirection/OOM error, 2 syntax or usage error, 126 not executable, 127 not found, 128+n killed by signal n.

### 11.3 Error Catalogue

| Code     | Condition                                                | Detected in | User message (example)                   | \$?       | Recovery                                    |
|----------|----------------------------------------------------------|-------------|------------------------------------------|-----------|---------------------------------------------|
| E-SYN-01 | Unterminated quote                                       | C-02        | syntax error: unterminated quote         | 2         | Line discarded                              |
| E-SYN-02 | Empty pipe side / missing redirect target / & not at end | C-04        | syntax error near                        | 2         | Line discarded                              |
| E-EXE-01 | Command not found                                        | C-05        | foo: command not found                   | 127       | Child exits 127                             |
| E-EXE-02 | Permission denied on program                             | C-05        | ./x: Permission denied                   | 126       | Child exits 126                             |
| E-RED-01 | Cannot open redirect file                                | C-06        | out.txt: Permission denied               | 1         | Command skipped                             |
| E-SYS-01 | pipe() or fork() failure                                 | C-05        | fork: Resource temporarily unavailable   | 1         | Kill and reap started stages; free pipeline |
| E-JOB-01 | fg/bg with unknown job                                   | C-07        | fg: %3: no such job                      | 1         | No change                                   |
| E-BI-01  | Invalid built-in argument                                | C-08        | export: '1A': not a valid identifier     | 1 or 2    | No change                                   |
| E-HIS-01 | History file unreadable/unwritable                       | C-09        | warning: cannot load history             | unchanged | Continue with empty history                 |
| E-SEC-01 | setuid/setgid detected                                   | C-01        | refusing to run with elevated privileges | 1         | Shell exits                                 |
| E-MEM-01 | Allocation failure                                       | C-10        | out of memory                            | 1         | Command abandoned                           |

### 11.4 Resource Clean-up

On every error path the component frees what it allocated and closes descriptors it opened. Pipe descriptors are closed in the parent immediately after forking all stages; if a later fork fails, the already-created children are sent SIGKILL and reaped so no zombie or open pipe remains.

## 12. Design Decisions and Rationale

| Decision                                                | Rationale                                                                                                                                                                                | Alternatives                                                       |
|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| SIGCHLD handler only sets a flag                        | Async-signal-safety; avoids races on the job list                                                                                                                                        | Reaping inside handler (unsafe with malloc'd job list)             |
| Built-ins in pipelines or with & run in a child process | Consistent with bash; guarantees SEC-005                                                                                                                                                 | Reject built-ins in pipelines                                      |
| Alias values are re-lexed; \$VAR values are not         | Aliases are trusted text the user typed (alias ll='ls -l' must become two words); variable values may come from untrusted environment and are inserted as single literal words (SEC-002) | Treat aliases like variables (alias with arguments would not work) |
| No word splitting of expanded values                    | Simpler and safer; documented deviation from bash in FR-003                                                                                                                              | Bash IFS splitting                                                 |
| execute_pipeline() owns the Pipeline                    | One owner prevents leaks and double frees (NFR-002)                                                                                                                                      | REPL frees after call (breaks background jobs)                     |
| Alias expansion in the expander, first word only        | Simple, matches POSIX behaviour, easy to unit test                                                                                                                                       | Expansion before lexing (breaks quoting)                           |
| Ring buffer of 1,000 history entries                    | Bounded memory and fast start-up (NFR-008)                                                                                                                                               | Unbounded list                                                     |
| No line-editing library                                 | Out of scope; keeps the binary dependency-free                                                                                                                                           | GNU readline (GPL, extra dependency)                               |

## 13. Work Division

| Member             | Components owned                                                                                 | Requirements                           | Supporting responsibility |
|--------------------|--------------------------------------------------------------------------------------------------|----------------------------------------|---------------------------|
| Anshul Poovaiah K  | C-01 REPL, C-02 Lexer, C-03 Expander, env part of C-10; built-ins export/unset/env               | FR-001, FR-002, FR-003, part of FR-009 | API review                |
| Archana Shivakumar | C-04 Parser, C-05 Executor & Pipeline; script mode                                               | FR-004, FR-005, FR-012                 | Integration test harness  |
| Ashlesh R V        | C-06 Redirection, C-07 Job Control & Signals; built-ins jobs/fg/bg                               | FR-006, FR-007, FR-008, part of FR-009 | CI/CD and Docker          |
| Atishay Jain       | C-08 Built-ins framework, C-09 History & Alias; built-ins cd/pwd/echo/exit/history/alias/unalias | FR-009, FR-010, FR-011                 | Unit tests, Valgrind, RTM |

Planned source layout:

```
include/*.h
src/{main,repl,lexer,expander,parser,executor,redirect,jobs,signals,
    builtin,history,alias,env,util,error}.c
tests/unit/ tests/integration/ tests/negative/ tests/fuzz/
Makefile Dockerfile .github/workflows/ci.yml
```

**PES UNIVERSITY, BANGALORE**

Department of Computer Science and Engineering

JACKFRUIT MINI-PROJECT — UE24CS341A Software Engineering

# Test Plan

for

**Simple Shell — POSIX-Compliant Command Line Interpreter in C**

| Item                 | Details                                                                                                                             |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Project              | Simple Shell — POSIX-Compliant Command Line Interpreter in C                                                                        |
| Project ID           | SH-2026-10                                                                                                                          |
| Group                | Team 10 (G-10)                                                                                                                      |
| Team Members         | Anshul Poovaiah K – PES1UG24CS071 / Archana Shivakumar – PES1UG24CS078 / Ashlesh R V – PES1UG24CS088 / Atishay Jain – PES1UG24CS097 |
| Course               | UE24CS341A – Software Engineering, V Semester                                                                                       |
| Methodology          | Agile (Scrum, 6 sprints)                                                                                                            |
| Version              | 1.0                                                                                                                                 |
| Date                 | 3 October 2026                                                                                                                      |
| Standards basis      | IEEE Std 829-2008 (Software Test Documentation)                                                                                     |
| Test plan identifier | TP-SH-2026-10-v1.0                                                                                                                  |

# Revision History

| Name    | Date        | Reason for Changes                                                               | Version |
|---------|-------------|----------------------------------------------------------------------------------|---------|
| Team 10 | 03-Oct-2026 | Initial test plan and test case specification derived from SRS v1.1 and SAD v1.0 | 1.0     |

## Table of Contents

- 1. Introduction
- 2. Test Items
- 3. Features to be Tested
- 4. Features Not to be Tested
- 5. Approach (5.1 Security Validation)
- 6. Item Pass/Fail Criteria
- 7. Suspension Criteria and Resumption Requirements
- 8. Test Deliverables
- 9. Testing Tasks and Schedule
- 10. Environmental Needs
- 11. Responsibilities, Staffing and Training
- 12. Risks and Contingencies
- 13. Approvals
- 14. Test Case Specifications
- 15. Requirements-to-Test Traceability

# 1. Introduction

## 1.1 Purpose

This plan defines how Team 10 will verify and validate Simple Shell (SH-2026-10) against SRS v1.1 and SAD v1.0: what is tested, at which levels, with which tools, by whom, and when a test or release is considered passed.

## 1.2 Scope

All functional requirements FR-001 to FR-012, non-functional requirements NFR-001 to NFR-008 and security requirements SEC-001 to SEC-007 are in scope at unit, integration and system level, including security validation (Section 5.1). Test cases are specified in Section 14; Actual Result and Status columns are completed when each test is executed during the implementation sprints.

## 1.3 References

- Team 10 — SRS v1.1; SAD v1.0; Synopsis v1.0.
- IEEE Std 829-2008.
- PES University — Jackfruit Software Engineering Project Guidelines, UE24CS341A.

## 1.4 Definitions

| Term                        | Meaning                                                                                        |
|-----------------------------|------------------------------------------------------------------------------------------------|
| UT / IT / ST                | Unit / Integration / System test                                                               |
| PT, MEM, ZMB, NEG, QA, PORT | Non-functional test families: performance, memory, zombie, negative, code quality, portability |
| SEC-Tnn                     | Security validation test                                                                       |
| Baseline                    | GNU bash 5.1 output for the same command, used as the reference expected result where stated   |

# 2. Test Items

| Item                                  | Version / Source                       | Notes                                                                      |
|---------------------------------------|----------------------------------------|----------------------------------------------------------------------------|
| simple_shell binary                   | Git tag of each sprint (v0.1 ... v1.0) | Built with make (release) and make debug (-g -fsanitize=address,undefined) |
| Source modules C-01 ... C-10          | src/*.c, include/*.h                   | Unit-tested individually through their headers                             |
| Dockerfiles (Ubuntu 22.04, Debian 12) | repository root                        | Used for PORT-01 and all CI runs                                           |
| Documentation                         | SRS v1.1, SAD v1.0                     | Inputs to test design and traceability                                     |

# 3. Features to be Tested

| Feature                          | Requirements | Test cases                      |
|----------------------------------|--------------|---------------------------------|
| Interactive REPL and prompt      | FR-001       | ST-REPL-01                      |
| Tokenization with quotes/escapes | FR-002       | UT-LEX-01, UT-LEX-02, UT-LEX-03 |
| Variable and \$? expansion       | FR-003       | UT-EXP-01, UT-EXP-02            |
| External command execution       | FR-004       | IT-EXE-01, IT-EXE-02            |
| Pipelines                        | FR-005       | UT-PAR-01, IT-PIP-01            |
| Redirection                      | FR-006       | IT-RED-01, IT-RED-02            |

|                                         |                   |                      |
|-----------------------------------------|-------------------|----------------------|
| Background jobs                         | FR-007            | IT-JOB-01            |
| Signal forwarding / job stop and resume | FR-008            | ST-SIG-01, ST-SIG-02 |
| Built-ins                               | FR-009            | UT-BI-01, IT-BI-02   |
| History                                 | FR-010            | UT-HIS-01, IT-HIS-02 |
| Aliases                                 | FR-011            | UT-ALI-01            |
| Script mode                             | FR-012            | ST-BAT-01            |
| End-to-end session                      | FR-001–FR-011     | ST-E2E-01            |
| Performance and start-up                | NFR-001, NFR-008  | PT-01, PT-02         |
| Memory safety                           | NFR-002           | MEM-01               |
| Zombie reaping                          | NFR-003           | ZMB-01               |
| Robustness and error format             | NFR-004, NFR-007  | NEG-01               |
| Maintainability                         | NFR-005           | QA-01                |
| Portability                             | NFR-006           | PORT-01              |
| Security                                | SEC-001 – SEC-007 | SEC-T01 – SEC-T07    |

## 4. Features Not to be Tested

| Feature                                                  | Reason                                                             |
|----------------------------------------------------------|--------------------------------------------------------------------|
| Scripting control structures (if, for, while, functions) | Out of scope (SRS 1.3)                                             |
| Arrow-key line editing / tab completion                  | Out of scope (SRS 3.1)                                             |
| Correctness of external programs (ls, grep, ...)         | Not part of the product (SRS 2.6); only their invocation is tested |
| Network / remote execution                               | No communications interface (SRS 3.3)                              |
| Non-English locales and multibyte validation             | Out of scope (SRS 7)                                               |
| Behaviour on non-Linux POSIX systems (macOS, BSD)        | Operating environment is Ubuntu/Debian only (SRS 2.4)              |

## 5. Approach

### 5.1 Security Validation

Security validation verifies each security objective (SO-1 to SO-3) through the requirements that realise it. It combines dynamic testing, fuzzing, static checks and a manual security review, and is executed in every sprint from Sprint 3 and in full before release.

| Technique               | What it checks                                                                                           | Tool                                       | Requirement      | Test             |
|-------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------|------------------|------------------|
| Fuzzing with sanitizers | Memory corruption from crafted input ( $\geq$ 1,000 random lines up to 1 MB, binary bytes, deep quoting) | Custom fuzz script + ASan/UBSan build      | SEC-001          | SEC-T01          |
| Injection testing       | Metacharacters inside quotes/variables reach programs literally; canary file survives                    | Bash harness                               | SEC-002          | SEC-T02          |
| Source audit (static)   | No system(), popen(), sh -c; Cppcheck security warnings = 0                                              | grep in CI, Cppcheck, Clang-Tidy           | SEC-002, NFR-005 | SEC-T02, QA-01   |
| Permission tests        | Redirection cannot bypass file modes; target unchanged on failure                                        | Bash harness as non-root user              | SEC-003          | SEC-T03          |
| Privilege test          | Shell refuses setuid/setgid execution                                                                    | Docker container (root sets up, user runs) | SEC-004          | SEC-T04          |
| Isolation test          | Built-ins inside pipelines do not change shell state                                                     | Bash harness                               | SEC-005          | SEC-T05          |
| Confidentiality tests   | History file mode 0600; no descriptor leakage to children                                                | stat, /proc/self/fd                        | SEC-006, SEC-007 | SEC-T06, SEC-T07 |
| Manual security review  | Threat table in SAD 7.2 re-checked against code at each PR touching C-01, C-02, C-05, C-06, C-09         | GitHub PR review checklist                 | All SEC          | Review record    |

**Penetration-style test planning:** before release, one team member not owning the module under test acts as an attacker for one hour per component group (input handling C-01/C-02/C-03; execution C-05/C-06; state C-09), trying to crash the shell, read another user's history or escalate privileges. Findings are logged as GitHub issues labelled security and must be fixed and retested before v1.0.

### 5.2 Test Levels

| Level            | Scope                                                                                    | Method                                           | Tool                                    | Entry criterion                        |
|------------------|------------------------------------------------------------------------------------------|--------------------------------------------------|-----------------------------------------|----------------------------------------|
| Unit (UT)        | Single component through its header (lexer, expander, parser, built-ins, history, alias) | White-box; boundary and equivalence partitioning | Unity / Criterion, gcov for coverage    | Component compiles; header frozen      |
| Integration (IT) | Shell binary with real child processes, pipes and files                                  | Black-box; output compared with bash baseline    | tests/integration/run.sh (Bash harness) | Unit tests of involved components pass |
| System (ST)      | Complete shell in a real pty, including signals and script mode                          | Scenario-based, end-to-end                       | expect scripts in Docker                | Integration suite passes               |
| Non-functional   | Performance, memory, robustness, portability                                             | Measurement against SRS thresholds               | hyperfine/time, Valgrind, ps, CI matrix | Release candidate build                |

### 5.3 Test Design Techniques

- **Equivalence partitioning and boundary values:** e.g., pipeline length 1, 2, 16; history 999/1,000/1,001 entries; empty and 64 KiB lines.
- **Negative testing:** malformed syntax, missing files, missing permissions, unknown jobs (NEG-01 suite  $\geq$  30 cases).
- **Differential testing:** for deterministic commands, stdout/stderr/status compared with bash running the same input.

- **State-transition testing:** job state machine (SAD Fig. 9.5) covered by ST-SIG-01, ST-SIG-02, IT-JOB-01.

## 5.4 Coverage Goals

Unit-test line coverage  $\geq 80\%$  and branch coverage  $\geq 70\%$  for C-02, C-03, C-04, C-08 and C-09, measured with gcov/lcov in CI. 100% of FR, NFR and SEC requirements must have at least one executed test (Section 15).

## 5.5 Regression

All unit and integration tests run on every push and pull request in GitHub Actions; the full suite including Valgrind and the fuzz job runs nightly and before each sprint tag. A failing test blocks merging.

## 6. Item Pass/Fail Criteria

- A test case passes when every expected result in Section 14 is observed exactly; otherwise it fails and a defect is raised.
- A sprint build passes when 100% of unit tests and  $\geq 95\%$  of integration tests pass and no open Critical/High defect exists.
- The v1.0 release passes when 100% of test cases pass, NFR thresholds (SRS 6) are met, coverage goals (5.4) are met and all security findings are closed.

## 7. Suspension Criteria and Resumption Requirements

- Suspend system testing if the build fails to compile in CI, if  $> 20\%$  of integration tests fail, or if the shell crashes on start-up.
- Suspend security testing if the ASan build cannot be produced.
- Resume after the blocking defect is fixed, merged and the smoke subset (ST-REPL-01, IT-EXE-01, IT-PIP-01) passes.

## 8. Test Deliverables

- This Test Plan (with test case specifications).
- Unit-test and coverage reports (Icov HTML) per sprint.
- Integration/system test logs from CI.
- Valgrind and ASan reports.
- Security validation report and issue list.
- Defect log (GitHub Issues) and final Test Summary / Validation Report.

## 9. Testing Tasks and Schedule

| Sprint   | Testing tasks                                                                   | Test cases executed                                                                |
|----------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Sprint 1 | Set up Unity, CI workflow, Docker images; REPL and single-command tests         | ST-REPL-01, IT-EXE-01, IT-EXE-02, PORT-01                                          |
| Sprint 2 | Lexer, expander and built-in unit tests                                         | UT-LEX-01..03, UT-EXP-01..02, UT-BI-01, IT-BI-02                                   |
| Sprint 3 | Parser, pipeline and redirection tests; start security tests                    | UT-PAR-01, IT-PIP-01, IT-RED-01..02, SEC-T02, SEC-T03, SEC-T05                     |
| Sprint 4 | Job control and signal tests in pty                                             | IT-JOB-01, ST-SIG-01..02, ZMB-01                                                   |
| Sprint 5 | History/alias tests; Valgrind; fuzzing; static analysis                         | UT-HIS-01, IT-HIS-02, UT-ALI-01, MEM-01, SEC-T01, SEC-T04, SEC-T06, SEC-T07, QA-01 |
| Sprint 6 | Full regression, performance, script mode, end-to-end, penetration-style review | All, plus PT-01, PT-02, NEG-01, ST-BAT-01, ST-E2E-01                               |

## 10. Environmental Needs

- Docker images: Ubuntu 22.04 and Debian 12 with gcc 11+, make, valgrind, cppcheck, clang-tidy, expect, bash 5.1, Icov.
- Reference performance machine: 2 vCPU, 4 GB RAM container (NFR-001).
- A non-root test user tester and a second user other for permission tests.

- GitHub Actions runners for CI; local terminal (pty) for signal tests.

## 11. Responsibilities, Staffing and Training

| Member             | Testing responsibility                                                                     |
|--------------------|--------------------------------------------------------------------------------------------|
| Anshul Poovaiah K  | UT-LEX, UT-EXP, ST-REPL, SEC-T01, SEC-T02                                                  |
| Archana Shivakumar | UT-PAR, IT-EXE, IT-PIP, ST-BAT, PT-01, SEC-T05                                             |
| Ashlesh R V        | IT-RED, IT-JOB, ST-SIG, ZMB-01, PORT-01, SEC-T03, CI pipeline                              |
| Atishay Jain       | UT-BI, IT-BI, UT-HIS, IT-HIS, UT-ALI, MEM-01, PT-02, SEC-T04, SEC-T06, SEC-T07, RTM upkeep |
| All                | ST-E2E-01, NEG-01, QA-01; peer review of tests for modules they do not own                 |

Training: one session in Sprint 1 on Unity, Valgrind and expect, run by Atishay and Ashlesh.

## 12. Risks and Contingencies

| Risk                                               | Impact                       | Contingency                                                                                |
|----------------------------------------------------|------------------------------|--------------------------------------------------------------------------------------------|
| Signal/job-control tests flaky in CI (no real TTY) | False failures               | Run ST-SIG tests under expect with a pty; mark as required locally if CI pty is unreliable |
| Performance numbers vary between machines          | NFR-001 disputed             | Measure only in the reference container; report median of 5 runs                           |
| Late integration of modules                        | Integration tests start late | Header contracts frozen in Sprint 1; stubs used for unit tests                             |
| Valgrind slowdown on large suites                  | Long CI time                 | Run Valgrind nightly and on release branches only                                          |

## 13. Approvals

| Name               | Role                  | Signature / Date |
|--------------------|-----------------------|------------------|
| Anshul Poovaiah K  | Developer / Tester    |                  |
| Archana Shivakumar | Developer / Tester    |                  |
| Ashlesh R V        | Developer / CI owner  |                  |
| Atishay Jain       | Developer / Test lead |                  |
| Course Faculty     | Reviewer              |                  |

## 14. Test Case Specifications

36 test cases are specified: 22 functional (unit, integration and system) and 14 non-functional and security. Actual Result and Status are recorded when each test is executed in the sprint shown in Section 9; until then they read "Pending".

| ID        | Req.            | Level / Type | Description & Precondition                                        | Steps / Test Data                                                                  | Expected Result                                                                                                       | Actual  | Status  |
|-----------|-----------------|--------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------|---------|
| UT-LEX-01 | FR-002          | Unit         | Tokenize quotes and escapes. Pre: lexer built.                    | lex(echo "a b" 'c \$d' e\ f)                                                       | Tokens: [echo] [a b] [c \$d] [e f]; 4 WORD tokens                                                                     | Pending | Not run |
| UT-LEX-02 | FR-002, NFR-004 | Unit         | Unterminated quote is a syntax error.                             | lex(echo "abc)                                                                     | Returns NULL, err = ERR_SYNTAX; no crash                                                                              | Pending | Not run |
| UT-LEX-03 | FR-002          | Unit         | Operators recognised without spaces.                              | lex(ls>o 2>e wc&)                                                                  | WORD(ls) GREAT WORD(o) ERR_GREAT WORD(e) PIPE WORD(wc) AMP                                                            | Pending | Not run |
| UT-EXP-01 | FR-003          | Unit         | Variable expansion. Pre: HOME=/home/t, UNDEF unset.               | expand tokens of echo \$HOME \$UNDEF '\$HOME'                                      | argv = [echo, /home/t, \$HOME] — 3 args: \$UNDEF produces no argument; the single-quoted \$HOME stays literal         | Pending | Not run |
| UT-EXP-02 | FR-003          | Unit         | \$? expansion. Pre: ctx.last_status = 1.                          | expand echo \$?                                                                    | argv = [echo, 1]                                                                                                      | Pending | Not run |
| UT-PAR-01 | FR-005, FR-006  | Unit         | Parse pipeline with redirections.                                 | parse tokens of cat < in   sort -r >> out &                                        | 2 commands; cmd1.input_file=in; cmd2.output_file=out, append=1; is_background=1                                       | Pending | Not run |
| IT-EXE-01 | FR-004          | Integration  | Run an external command. Pre: shell running.                      | ls / then echo \$?                                                                 | Output identical to bash ls /; then 0                                                                                 | Pending | Not run |
| IT-EXE-02 | FR-004, NFR-007 | Integration  | Unknown and non-executable commands.                              | 1) nosuchcmd 2) ./data.txt (mode 0644)                                             | 1) stderr simple_shell: nosuchcmd: command not found, \$? = 127; 2) Permission denied, \$? = 126; shell still running | Pending | Not run |
| IT-PIP-01 | FR-005          | Integration  | Multi-stage and 16-stage pipelines.                               | 1) seq 1 100000   grep 7   sort -n   tail -12) echo hi piped through 15 cat stages | 1) 99997 (same as bash), \$? = 0 2) hi; completes in < 1 s; ls /proc/<shell pid>/fd shows only 0,1,2 afterwards       | Pending | Not run |
| IT-RED-01 | FR-006          | Integration  | All redirection operators.                                        | echo a > f; echo b >> f; cat < f; ls /nope 2> e; ls / /nope &> both                | f contains a,b; cat prints a,b; e contains the ls error; both contains stdout and stderr; no output on terminal       | Pending | Not run |
| IT-RED-02 | FR-006, SEC-003 | Integration  | Redirect to unwritable file. Pre: ro.txt mode 0444 contains keep. | echo x > ro.txt; echo \$?                                                          | stderr simple_shell: ro.txt: Permission denied; 1; ro.txt still contains keep; echo not executed                      | Pending | Not run |
| IT-JOB-01 | FR-007          | Integration  | Background job.                                                   | sleep 2 &, immediately jobs, wait 3 s, press Enter                                 | [1] <pid> printed and prompt returns < 100 ms; jobs shows [1]+ Running sleep 2; then [1]+ Done sleep 2                | Pending | Not run |
|           | FR-008          | System       |                                                                   |                                                                                    |                                                                                                                       | Pending |         |

|            |                        |              |                                                                     |                                                                                                                                                                                         |                                                                                                                                                                                                                      |         |         |
|------------|------------------------|--------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|---------|
| ST-SIG-01  |                        |              | Ctrl+C reaches only the child. Pre: interactive pty (expect).       | Run <code>sleep 30</code> , send Ctrl+C, run <code>echo \$?</code>                                                                                                                      | sleep terminated; shell alive with new prompt; 130                                                                                                                                                                   |         | Not run |
| ST-SIG-02  | FR-008, FR-007, FR-009 | System       | Stop and resume a job.                                              | Run <code>sleep 30</code> , send Ctrl+Z, jobs, bg %1, jobs, fg %1, Ctrl+C                                                                                                               | [1]+ Stopped sleep 30; state Stopped → Running after bg; fg returns it to foreground; Ctrl+C ends it; shell alive                                                                                                    | Pending | Not run |
| UT-BI-01   | FR-009                 | Unit         | cd variants. Pre: HOME=/tmp/h exists.                               | builtin cd /usr; cd; cd -; cd ~; cd /nonexistent                                                                                                                                        | cwd=/usr; cwd=/tmp/h; cwd=/usr; cwd=/tmp/h; last returns 1 with error, cwd unchanged                                                                                                                                 | Pending | Not run |
| IT-BI-02   | FR-009                 | Integration  | Built-ins run in parent.                                            | <code>export X=5; env   grep ^X=; unset X; echo [\$X]; cd /tmp; pwd; exit 3</code>                                                                                                      | X=5; []; /tmp; shell process exits with status 3                                                                                                                                                                     | Pending | Not run |
| UT-HIS-01  | FR-010                 | Unit         | Ring buffer boundary.                                               | <code>history_add 1,001 lines c1..c1001</code>                                                                                                                                          | count = 1,000; first entry c2; last c1001                                                                                                                                                                            | Pending | Not run |
| IT-HIS-02  | FR-010, SEC-006        | Integration  | History persists across sessions. Pre: no history file.             | Session 1: <code>echo one; exit</code> ; Session 2: <code>history</code>                                                                                                                | File created; <code>stat -c %a = 600</code> ; session 2 lists <code>echo one</code>                                                                                                                                  | Pending | Not run |
| UT-ALI-01  | FR-011                 | Unit         | Alias set, expand, guard, remove.                                   | <code>alias ll='ls -l'; expand ll /; alias ls='ls -F'; expand ls; unalias ll</code>                                                                                                     | <code>ls -l /; ls -F</code> (expanded once, no loop); ll removed, <code>alias_get(ll)=NULL</code>                                                                                                                    | Pending | Not run |
| ST-REPL-01 | FR-001                 | System       | Prompt format and long line. Pre: user tester on host box, cwd ~/w. | Start shell; enter a 64 KiB echo line                                                                                                                                                   | Prompt <code>tester@box:~/w\$</code> ; full 64 KiB echoed; prompt re-shown                                                                                                                                           | Pending | Not run |
| ST-BAT-01  | FR-012                 | System       | Script mode.                                                        | <code>simple_shell s.sh</code> where <code>s.sh = echo a, false, echo b, exit 4</code> ; also <code>printf 'echo z\n'   simple_shell</code>                                             | Output a b with no prompt; exit status 4; second prints z only; no history file written                                                                                                                              | Pending | Not run |
| ST-E2E-01  | FR-001-<br>FR-011      | System       | Realistic session.                                                  | Pre: empty dir ~/e2e. Input: <code>cd ~/e2e; alias ll='ls -l'; export N=3; seq 1 \$N   sort -r &gt; o.txt; cat &lt; o.txt   wc -l; ll o.txt; sleep 1 &amp; jobs; history; exit 0</code> | 3; ll lists o.txt; o.txt contains 3, 2, 1; [1] <pid> then jobs shows Running; history lists all lines in order; no stderr output; exit status 0. Outputs match bash for these commands (none rely on word splitting) | Pending | Not run |
| PT-01      | NFR-001                | Performance  | Dispatch overhead. Pre: reference container.                        | Run 1,000-line /bin/true script through shell vs C baseline harness, 5 runs                                                                                                             | Median (shell – baseline)/1,000 ≤ 2.0 ms                                                                                                                                                                             | Pending | Not run |
| PT-02      | NFR-008                | Performance  | Start-up with full history.                                         | 1,000-entry history file; measure time to first prompt, 20 runs                                                                                                                         | Median ≤ 100 ms                                                                                                                                                                                                      | Pending | Not run |
| MEM-01     | NFR-002                | Memory       | Leak check.                                                         | <code>valgrind --leak-check=full --error-exitcode=1 ./simple_shell regression_1000.sh</code>                                                                                            | 0 bytes definitely/indirectly lost; 0 errors; exit code ≠ valgrind error code                                                                                                                                        | Pending | Not run |
| ZMB-01     | NFR-003                | Reliability  | Zombie reaping.                                                     | Start 50 <code>sleep 0.1 &amp; jobs</code> ; sample <code>ps -o stat --ppid &lt;shell&gt;</code> every 50 ms for 2 s                                                                    | No process with state Z observed for > 100 ms                                                                                                                                                                        | Pending | Not run |
| NEG-01     | NFR-004, NFR-007       | Robustness   | Negative suite (≥ 30 inputs).                                       | Run tests/negative/*.in (bad quotes,   ls, ls >, missing files, 100 KB line, fg %9, ...)                                                                                                | Shell survives 100%; each error on stderr prefixed <code>simple_shell: ;</code> prompt re-shown                                                                                                                      | Pending | Not run |
| QA-01      | NFR-005                | Code quality | Modularity and static analysis.                                     | <code>nm -g</code> on objects; <code>cppcheck --enable=all src/</code>                                                                                                                  | 0 non-static globals outside signals.o; 0 Cppcheck errors                                                                                                                                                            | Pending | Not run |
| PORT-01    | NFR-006                | Portability  | Build matrix.                                                       |                                                                                                                                                                                         |                                                                                                                                                                                                                      | Pending |         |

|         |         |          |                                                                          |                                                                                                                            |                                                                                                                       |         |         |
|---------|---------|----------|--------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------|---------|
|         |         |          |                                                                          | CI builds with -Werror in Ubuntu 22.04 and Debian 12; run integration suite                                                | 0 warnings; 100% integration pass on both                                                                             |         | Not run |
| SEC-T01 | SEC-001 | Security | Fuzzing with ASan/UBSan.                                                 | Feed 1,000 random lines ( $\leq$ 1 MB, random bytes and quotes) to debug build                                             | 0 sanitizer reports; shell never crashes                                                                              | Pending | Not run |
| SEC-T02 | SEC-002 | Security | Command injection. Pre: /tmp/canary exists.                              | echo "x; rm /tmp/canary"; export V='\$(rm /tmp/canary)'; echo \$V; search src/ for the strings system(, popen( and /bin/sh | Prints x; rm /tmp/canary and then \$(rm /tmp/canary) literally; /tmp/canary still exists; the search finds no matches | Pending | Not run |
| SEC-T03 | SEC-003 | Security | Permission bypass. Pre: /home/other/secret owned by other, mode 0600.    | As tester: cat < /home/other/secret; echo x > /home/other/secret                                                           | Both fail with Permission denied, \$? = 1; file unchanged                                                             | Pending | Not run |
| SEC-T04 | SEC-004 | Security | Refuse setuid. Pre: binary chown other, chmod u+s.                       | Run as tester                                                                                                              | simple_shell: refusing to run with elevated privileges; exit status 1                                                 | Pending | Not run |
| SEC-T05 | SEC-005 | Security | Environment isolation in pipelines and background jobs. Pre: cwd = /tmp. | 1) export Z=1   cat; echo [\$Z] 2) cd /   cat; pwd 3) export W=2 & cd / & wait 1s; echo [\$W]; pwd                         | 1) [] 2) /tmp 3) [] and /tmp — shell state unchanged in all cases                                                     | Pending | Not run |
| SEC-T06 | SEC-006 | Security | History permissions corrected. Pre: history file mode 0644.              | Start shell; stat -c %a ~/.simple_shell_history                                                                            | Warning printed; mode now 600                                                                                         | Pending | Not run |
| SEC-T07 | SEC-007 | Security | No descriptor leakage.                                                   | Interactive session: ls -l /proc/self/fd                                                                                   | Only 0, 1, 2 and ls's own directory fd; no entry for .simple_shell_history or pipes                                   | Pending | Not run |

## 14.1 Test Data Matrix

| Scenario               | Data                     | Expected                           |
|------------------------|--------------------------|------------------------------------|
| Valid                  | ls -l   wc -l            | Same count as bash, \$? = 0        |
| Invalid                | ls                       | Syntax error, \$? = 2              |
| Boundary (pipeline)    | 16-stage cat pipeline    | Completes; no leaked descriptors   |
| Boundary (history)     | 1,001 entries            | Oldest dropped; 1,000 kept         |
| Boundary (line length) | 64 KiB / 1 MB line       | Accepted; no ASan errors           |
| Security               | echo "a; rm /tmp/canary" | Literal output; canary kept        |
| Failure                | Redirect to 0444 file    | Permission denied, command not run |
| Signal                 | Ctrl+Z on foreground job | Job STOPPED, shell alive           |

## 15. Requirements-to-Test Traceability

Every requirement in SRS v1.1 is covered by at least one test case (27 of 27). The full chain from requirement to architecture, design and code is in SRS Appendix C.

| Requirement | Test case(s)                               | Coverage |
|-------------|--------------------------------------------|----------|
| FR-001      | ST-REPL-01, ST-E2E-01                      | Covered  |
| FR-002      | UT-LEX-01, UT-LEX-02, UT-LEX-03, ST-E2E-01 | Covered  |
| FR-003      | UT-EXP-01, UT-EXP-02, ST-E2E-01            | Covered  |
| FR-004      | IT-EXE-01, IT-EXE-02, ST-E2E-01            | Covered  |
| FR-005      | UT-PAR-01, IT-PIP-01, ST-E2E-01            | Covered  |
| FR-006      | UT-PAR-01, IT-RED-01, IT-RED-02, ST-E2E-01 | Covered  |
| FR-007      | IT-JOB-01, ST-SIG-02                       | Covered  |
| FR-008      | ST-SIG-01, ST-SIG-02                       | Covered  |
| FR-009      | UT-BI-01, IT-BI-02, ST-SIG-02, ST-E2E-01   | Covered  |
| FR-010      | UT-HIS-01, IT-HIS-02, ST-E2E-01            | Covered  |
| FR-011      | UT-ALI-01, ST-E2E-01                       | Covered  |
| FR-012      | ST-BAT-01                                  | Covered  |
| NFR-001     | PT-01                                      | Covered  |
| NFR-002     | MEM-01                                     | Covered  |
| NFR-003     | ZMB-01                                     | Covered  |
| NFR-004     | UT-LEX-02, NEG-01                          | Covered  |
| NFR-005     | QA-01                                      | Covered  |
| NFR-006     | PORT-01                                    | Covered  |
| NFR-007     | IT-EXE-02, NEG-01                          | Covered  |
| NFR-008     | PT-02                                      | Covered  |
| SEC-001     | SEC-T01                                    | Covered  |
| SEC-002     | SEC-T02                                    | Covered  |
| SEC-003     | IT-RED-02, SEC-T03                         | Covered  |
| SEC-004     | SEC-T04                                    | Covered  |
| SEC-005     | SEC-T05                                    | Covered  |
| SEC-006     | IT-HIS-02, SEC-T06                         | Covered  |
| SEC-007     | SEC-T07                                    | Covered  |